ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

BÁO CÁO BÀI TẬP LỚN

PHÁT TRIỂN ỨNG DỤNG IOT

Chủ đề:

Giảng viên hướng dẫn	**TS. Lê Trọng Nhân**	
Sinh viên thực hiện 1	**Đoàn Trần Cao Trí**	**2470510**
Sinh viên thực hiện 2	**Đặng Nguyễn Tấn Tài**	**2570357**
Sinh viên thực hiện 3	**Dương Lễ**	**2470102**

Lớp: CH01

Thành phố Hồ Chí Minh, 04/2026

Mục lục

I Bối cảnh và phát biểu bài toán chuỗi lạnh vaccine

1 Introduction và Problem Statement 1

1.1 Bối cảnh nghiệp vụ 1

1.2 Tầm nhìn sản phẩm 1

1.3 Phạm vi sản phẩm 1

1.4 Đối tượng sử dụng 2

1.5 Phát biểu bài toán 2

1.6 Pain points kỹ thuật 2

1.7 Tác động và cơ hội 2

II Yêu cầu hệ thống cho Vaccine Cold Chain Monitoring

2 Yêu cầu hệ thống và định hướng thiết kế 4

2.1 Định hướng thiết kế 4

2.2 Yêu cầu chức năng 4

2.3 Yêu cầu phi chức năng 4

2.4 Success criteria mức hệ thống 5

III Yêu cầu và định hướng thiết kế

3 Yêu cầu hệ thống và định hướng thiết kế 6

3.1 Định hướng thiết kế 6

3.2 Yêu cầu chức năng 6

3.3 Yêu cầu phi chức năng 6

3.4 Lý do chọn kiến trúc nhiều sender – một receiver 7

IV Kiến trúc và thiết kế các khối

4 Kiến trúc tổng thể và phân rã hệ thống 8

4.1 Bức tranh tổng quát của hệ thống. 8

4.2 Mô hình data plane và control plane 8

4.3 Lợi ích của việc phân rã module 9

4.4 Bảng so sánh phần cứng và phần mềm của hai khối 10

5 Thiết kế chi tiết khối Sender 10

5.1 Vai trò của sender trong toàn hệ thống 10

5.2 Luồng khởi tạo sender 11

5.3 Mô phỏng dữ liệu sensor 11

5.4 TinyML inference ở edge 12

5.5 Lớp mạng và sender transport 12

5.6 Giao diện web cục bộ của sender 13

6 Thiết kế chi tiết khối Receiver 16

6.1 Receiver như một gateway thực thụ. 16

6.2 Luồng khởi tạo receiver 17

6.3 Mô hình dữ liệu trung tâm của receiver 18

6.4 Known sender và unknown device 19

6.5 Cơ chế online/offline 20

6.6 Web UI quản trị của receiver. 21

V Giao thức, luồng điều khiển và tích hợp cloud

7 Thiết kế giao thức truyền thông 25

7.1 Lý do chọn TCP nội bộ 25

7.2 Định dạng packet TCP 25

7.3 Chi tiết packet telemetry 26

7.4 Chi tiết packet config và ACK 26

7.5 Giao thức MQTT với CoreIoT 26

8 Thiết kế luồng điều khiển cấu hình và RPC 27

8.1 Tại sao cần đồng bộ cấu hình hai chiều 27

8.2 Luồng đổi threshold từ receiver UI 28
8.3 Luồng đổi threshold từ RPC cloud 28
8.4 Retry và timeout 28
9 Tích hợp CoreIoT và vai trò của cloud 29
9.1 Receiver như một MQTT gateway. 29
9.2 Giá trị của tầng cloud đối với hệ thống 30
9.3 Ý nghĩa của việc hỗ trợ RPC 32

VI Ràng buộc, độ tin cậy và phân tích mã nguồn

10 Ràng buộc kỹ thuật và quyết định thiết kế. 33
10.1 Bảng hằng số hệ thống và ý nghĩa vận hành 33
10.2 Trade-off của các quyết định kiến trúc 34
11 Tình huống lỗi, cơ chế phòng vệ và bằng chứng kỹ thuật 34
11.1 Các tình huống lỗi điển hình 34
11.2 Cơ chế phòng vệ trong mã nguồn 34
11.3 Bảng đối chiếu luận điểm và bằng chứng mã nguồn 35
11.4 Ghi chú hình cần chèn ở chương này 35
12 Phân tích mã nguồn theo module và theo dòng ý nghĩa 36
12.1 Tổ chức phân tích mã nguồn 36
12.2 Trích đoạn 2: suy luận TinyML trên sender 37
12.3 Trích đoạn 3: gửi telemetry qua TCP 37
12.4 Trích đoạn 4: xử lý telemetry ở receiver 38
12.5 Trích đoạn 5: gửi cấu hình từ receiver xuống sender. 39
12.6 Trích đoạn 6: publish RPC response lên CoreIoT. 40

VII Giao diện, đánh giá và kết luận

13 Kết quả thực nghiệm 41
13.1 Thiết lập thực nghiệm 41
14 Kịch bản thực nghiệm và kết quả đánh giá 41
14.1 Mục tiêu và kết quả tổng quát 41
14.2 Danh sách test case và kết quả. 42

14.3 Bằng chứng thực nghiệm: cảnh báo AI và trạng thái thiết bị 43
14.4 Đánh giá ưu điểm của hệ thống 44
14.5 Hạn chế hiện tại 44

VIII Hướng phát triển và kết luận

15 Vì sao kiến trúc này phù hợp bối cảnh triển khai hiện tại 45
16 Khả năng mở rộng trong tương lai 45
17 Khả năng áp dụng thực tế 45
18 Kết luận 46

IX Tài liệu tham khảo

19 Nguồn tham khảo 47

X Phụ lục

1 Phụ lục A: Trích đoạn mã nguồn tiêu biểu và diễn giải ngắn 49

Danh sách hình vẽ

1 Bức tranh tổng thể cho bài toán Vaccine Cold Chain Monitoring: nhiều điểm đo (sender) gửi telemetry về một gateway receiver, sau đó đồng bộ cloud để giám sát, cảnh báo và truy vết. 3

2 Sơ đồ mạng nội bộ cho kịch bản chuỗi lạnh vaccine: receiver vừa phát SoftAP phục vụ vận hành tại chỗ vừa kết nối hạ tầng Wi-Fi, sender gửi telemetry về gateway để hợp nhất và đẩy cloud. 5

3 Sơ đồ tách *data plane* và *control plane* giúp nhận diện rõ hai loại luồng trong hệ thống. 9

4 Kiến trúc module của sender gồm các khối trạng thái, mô phỏng, AI, transport, lưu trữ và giao diện web. 9

5 Kiến trúc module của receiver gồm runtime, transport, cloud gateway, lưu trữ và giao diện web. 10

6 Flowchart phần khởi tạo và vòng lặp sender. Sơ đồ cho thấy sender liên tục mô phỏng cảm biến, chạy AI và duy trì mạng. 11

7 Pipeline TinyML trên sender: từ dữ liệu mô phỏng sang tensor input, qua bộ thông dịch TFLite Micro, rồi trả ra điểm bất thường và nhãn trạng thái. 12

8 Mô hình logic của TFLite được sử dụng ở mức báo cáo: hai đầu vào đặc trưng và một đầu ra điểm bất thường. 12

9 Cơ chế WebSocket snapshot của sender và receiver, cho phép web UI nhận trạng thái thời gian thực mà không cần tải lại trang. 13

10 Giao diện web sender ESP32-01 trong thực nghiệm. 14

11 Giao diện web sender ESP32-02 trong thực nghiệm. 15

12 Dialog cài đặt Wi-Fi và receiver của sender. 16

13 Cấu trúc lưu trữ LittleFS dùng chung cho cả sender và receiver: tệp cấu hình, danh sách sender đã đăng ký và giao diện web tĩnh. 17

14 Flowchart phần khởi tạo và vòng lặp receiver. Sơ đồ cho thấy receiver đồng thời xử lý DNS, TCP, Wi-Fi, MQTT và snapshot cho UI. 18

15 Mô hình dữ liệu trung tâm của receiver gồm cấu hình gateway, trạng thái sender đã đăng ký, thiết bị lạ và các kết nối TCP đang sống. 19

16 Flowchart phân loại telemetry theo mã thiết bị: sender đã đăng ký được cập nhật vào bảng trạng thái, còn sender chưa đăng ký được đưa vào danh sách thiết bị lạ. 20
17 Flowchart chuyển trạng thái online/offline của sender trên receiver dựa vào timeout. 20
18 Giao diện receiver: tổng quan 2 sender online và trạng thái CoreIoT. 21
19 Chi tiết sender ESP32-01 trên giao diện receiver — thông tin cơ bản. 22
20 Chi tiết sender ESP32-01 — vị trí, cấu hình và kết nối TCP. 23
21 Dialog cài đặt receiver và CoreIoT. 24
22 Bảng tổng hợp ba loại message TCP và các trường dữ liệu chính được trao đổi giữa sender và receiver. 25
23 Sequence diagram mô tả nhiều sender gửi telemetry lên receiver, receiver cập nhật trạng thái và tiếp tục publish lên cloud. 26
24 Cấu trúc phân cấp topic MQTT Gateway dùng cho tích hợp CoreIoT. 27
25 Sequence diagram mô tả receiver đại diện sender khi gửi connect, telemetry, attribute và xử lý RPC qua MQTT gateway. 27
26 Sequence diagram mô tả quá trình receiver gửi packet cấu hình xuống sender và nhận lại gói xác nhận. Cùng cơ chế này cũng được dùng khi có RPC từ cloud. 28
27 State machine đồng bộ cấu hình giữa receiver và sender. 29
28 Flowchart cơ chế retry cấu hình của receiver khi sender chưa ACK trong thời gian cho phép. 29
29 Gateway dangtai trên CoreIoT với 2 thiết bị ESP32 đang hoạt động. 30
30 Danh sách thiết bị CoreIoT sau khi receiver đăng ký hai sender. 31
31 Dashboard CoreIoT: telemetry và trạng thái AI của hai sender. 31
32 Telemetry ESP32-02 trên CoreIoT — ai_score ANOMALY và GPS. 32
33 Dashboard chi tiết thiết bị CoreIoT: gauge, biểu đồ lịch sử và threshold. 32
34 Sơ đồ state machine đồng bộ cấu hình: pending, awaiting ACK, timeout, retry, success/fail. 35
35 Sequence xử lý lỗi RPC khi sender offline và phản hồi lại cloud. 36
36 Flowchart phân bổ slot TCP nhiều sender và cơ chế chống starvation. 36
37 Sequence end-to-end: dữ liệu telemetry đi lên từ sender qua receiver lên cloud, lệnh điều khiển đi ngược chiều và được xác nhận bằng ACK. Đây là luồng đã được xác nhận trong thực nghiệm. 41

38 Hệ thống cảnh báo CoreIoT xác nhận kết quả TC5 (offline/status) và TC7 (AI anomaly). 43

Phần I

Bối cảnh và phát biểu bài toán chuỗi lạnh vaccine

1 Introduction và Problem Statement

1.1 Bối cảnh nghiệp vụ

Trong vận hành tiêm chủng, chất lượng vaccine phụ thuộc trực tiếp vào việc duy trì nghiêm ngặt chuỗi lạnh (*cold chain*) từ kho bảo quản đến điểm sử dụng. Theo bài tổng hợp của VNVC, phần lớn vaccine được bảo quản trong dải 2–8°C (mức trung bình ưu tiên khoảng 5°C); một số loại cần dải đông lạnh −15– − 25°C, và một số vaccine đặc thù yêu cầu âm sâu như −40°C hoặc −70°C. Chỉ một sai lệch ngắn ngoài ngưỡng cho phép cũng có thể làm giảm hiệu lực sinh học.

Ở quy mô vận hành nhiều tủ lạnh/tủ đông, dữ liệu nhiệt độ và độ ẩm biến thiên theo thời gian thực, trong khi thao tác mở cửa, xuất/nhập hàng và dao động tải nhiệt diễn ra liên tục. Vì vậy, mô hình theo dõi định kỳ bằng tay khó phát hiện các spike ngắn hoặc sự cố cục bộ xảy ra ngoài khung kiểm tra.

1.2 Tầm nhìn sản phẩm

Hệ thống hướng tới một nền tảng *IoT-based Vaccine Cold Chain Monitoring* với bốn mục tiêu cốt lõi: giám sát realtime nhiệt độ/độ ẩm/trạng thái thiết bị, cảnh báo sớm khi vượt ngưỡng rủi ro, lưu vết dữ liệu đầy đủ phục vụ audit–compliance, và giảm tải vận hành thủ công thông qua tự động hóa quan sát.

1.3 Phạm vi sản phẩm

Giải pháp bao gồm ba lớp chính. Lớp thiết bị hiện trường gồm sensor node (sender) và gateway (receiver). Lớp xử lý dữ liệu gồm ingestion, chuẩn hóa packet, quản lý trạng thái và đẩy dữ liệu lên nền tảng cloud. Lớp ứng dụng gồm dashboard giám sát, cảnh báo và các luồng điều khiển cấu hình từ xa.

Trong phạm vi tài liệu này, hệ thống được mô tả theo kiến trúc nhiều sender – một receiver để phản ánh vận hành đa tủ/lô điểm đo. Dữ liệu đầu vào hiện có thể được mô phỏng cho mục đích kiểm thử pipeline, nhưng toàn bộ luồng kỹ thuật được thiết kế tương thích khi thay bằng cảm biến thật.

1.4 Đối tượng sử dụng

- Nhân sự vận hành kho/lab vaccine cần theo dõi điều kiện bảo quản theo thời gian thực.
- Quản lý cơ sở cần cái nhìn tập trung và trạng thái cảnh báo tức thời.
- Bộ phận QA/Compliance cần dữ liệu liên tục, truy vết được và khó bị chỉnh sửa.
- Cấp điều hành cần báo cáo tổng hợp rủi ro vận hành và mức độ tuân thủ.

1.5 Phát biểu bài toán

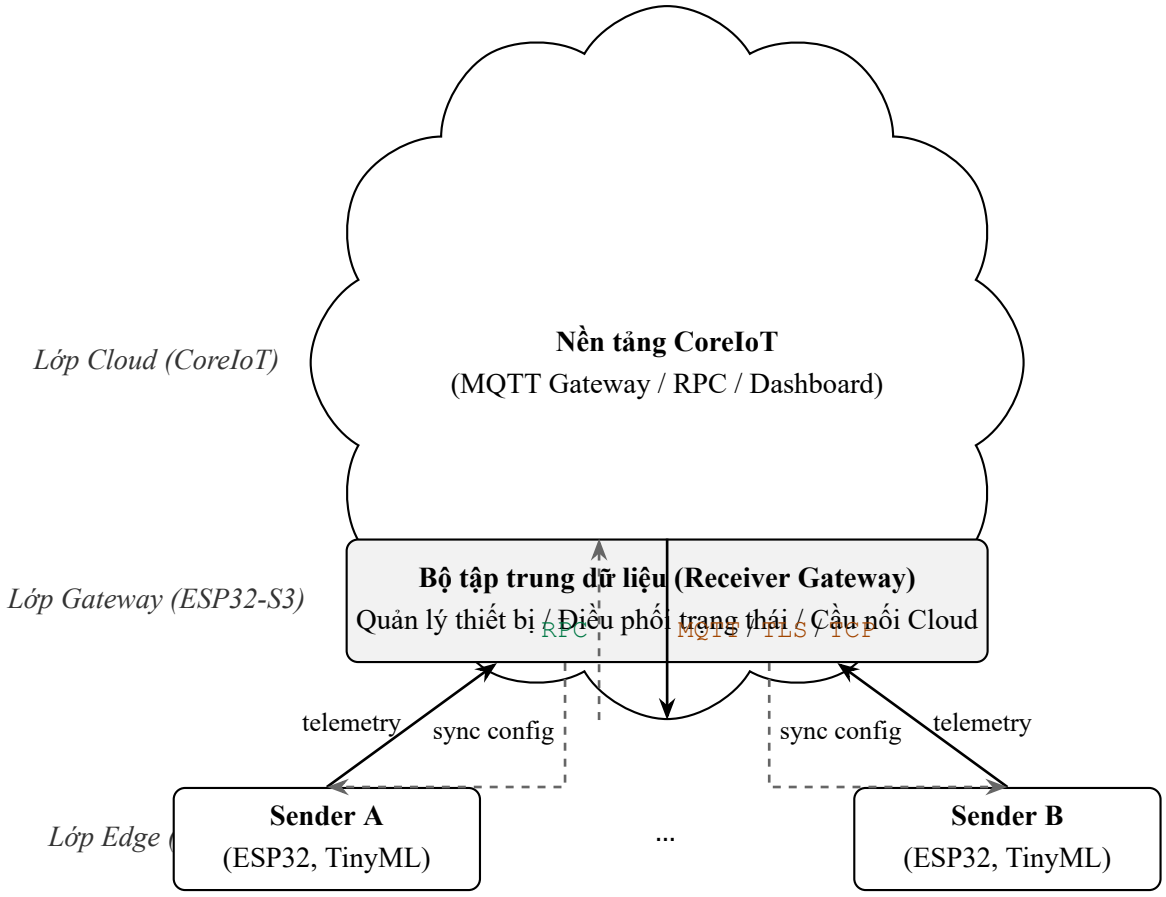
Vấn đề cốt lõi là nhiều điểm bảo quản vẫn dựa vào kiểm tra theo ca hoặc ghi chép rời rạc, dẫn đến khoảng trống quan sát giữa các lần kiểm tra. Khi xảy ra sự cố như mất điện cục bộ, cửa mở lâu hoặc suy giảm hiệu suất làm lạnh, hệ thống thiếu cơ chế cảnh báo tức thời và thiếu chuỗi bằng chứng dữ liệu để truy nguyên.

1.6 Pain points kỹ thuật

1) **Thiếu giám sát realtime liên tục**: khó bắt được các biến động đột ngột (spike) ngắn nhưng ảnh hưởng chất lượng vaccine.
2) **Thiếu cảnh báo tức thời**: độ trễ phát hiện cao làm tăng nguy cơ hỏng lô.
3) **Truy vết mở cửa và sai lệch vận hành còn yếu**: khó xác định nguyên nhân gốc khi có sự cố.
4) **Audit trail không đầy đủ**: dữ liệu phân tán/đứt đoạn gây rủi ro compliance.
5) **Chi phí vận hành thủ công cao**: phụ thuộc vào nhân sự kiểm tra định kỳ và ghi chép tay.
6) **Thiếu năng lực dự đoán**: chưa có cơ chế nhận diện xu hướng bất thường trước khi lỗi xảy ra.

1.7 Tác động và cơ hội

Nếu không cải thiện, rủi ro bao gồm thất thoát tài chính do hỏng vaccine, gián đoạn vận hành, tăng rủi ro pháp lý và giảm độ tin cậy hệ thống tiêm chủng. Ngược lại, triển khai nền tảng IoT giám sát chuỗi lạnh tạo ra cơ hội nâng mức an toàn vận hành, tự động hóa kiểm soát, và xây dựng nền dữ liệu phục vụ phân tích tối ưu dài hạn.



Hình 1: Bức tranh tổng thể cho bài toán Vaccine Cold Chain Monitoring: nhiều điểm đo (sender) gửi telemetry về một gateway receiver, sau đó đồng bộ cloud để giám sát, cảnh báo và truy vết.

Phần II

Yêu cầu hệ thống cho Vaccine Cold Chain Monitoring

2 Yêu cầu hệ thống và định hướng thiết kế

2.1 Định hướng thiết kế

Thiết kế ưu tiên ba nguyên tắc: (1) **phát hiện sớm** sai lệch điều kiện bảo quản, (2) **truy vết đầy đủ** phục vụ audit, và (3) **vận hành bền vững** ở quy mô nhiều điểm đo. Vì vậy kiến trúc được tổ chức theo mô hình sender–receiver–cloud thay vì để từng node kết nối rời rạc trực tiếp lên cloud.

Sender tập trung đo lường và tiền xử lý tại biên; receiver làm gateway tập trung trạng thái, điều phối cấu hình và đồng bộ cloud; cloud đảm nhiệm giám sát đa điểm, lưu trữ lịch sử và luồng điều khiển từ xa. Cách phân lớp này giảm độ phức tạp khi mở rộng số thiết bị và cải thiện khả năng kiểm soát rủi ro.

2.2 Yêu cầu chức năng

Các yêu cầu chức năng chính của hệ thống cho bài toán chuỗi lạnh vaccine gồm:

- Mỗi sender phải thu thập (hoặc mô phỏng để kiểm thử) nhiệt độ, độ ẩm và timestamp dưới dạng telemetry.
- Sender phải đánh giá trạng thái cục bộ theo ngưỡng và sinh các chỉ báo bất thường (`ai_score`, `ai_status`).
- Receiver phải nhận telemetry từ nhiều sender qua TCP và quản lý trạng thái tập trung theo `device_id`.
- Receiver phải hỗ trợ đồng bộ cấu hình hai chiều (ngưỡng, metadata) và xác nhận qua ACK.
- Hệ thống phải có giao diện cục bộ để quan sát realtime và thao tác vận hành cơ bản.
- Receiver phải đồng bộ dữ liệu, thuộc tính và RPC lên cloud qua MQTT gateway.
- Hệ thống phải lưu được chuỗi dữ liệu đủ để truy vết sự kiện vượt ngưỡng và hành động xử lý.

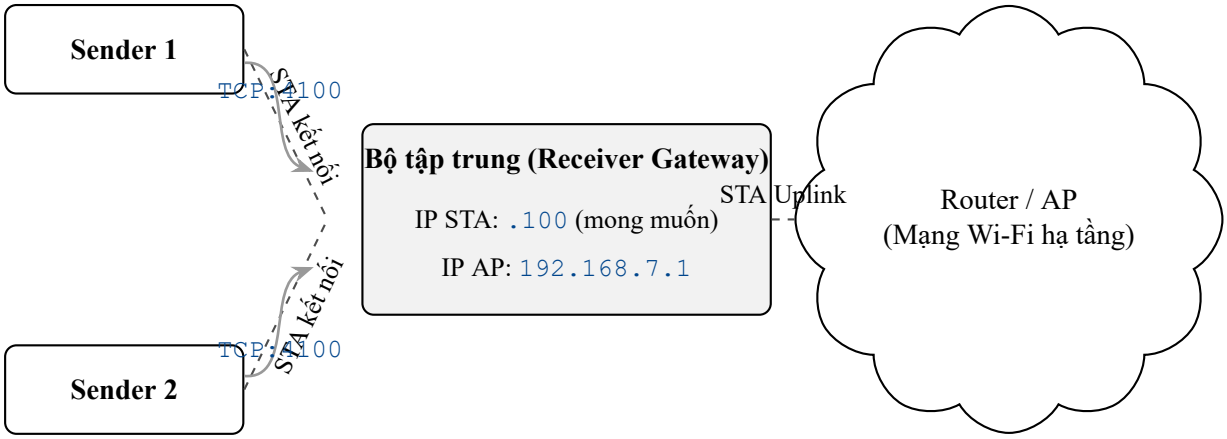
2.3 Yêu cầu phi chức năng

- **Realtime**: độ trễ cảnh báo mục tiêu thấp (ví dụ dưới 30 giây trong điều kiện mạng ổn định).
- **Reliability**: có timeout/retry và cơ chế online–offline rõ ràng để tránh mất trạng thái âm thầm.

- **Traceability**: dữ liệu phải nhất quán, liên tục theo thời gian và dễ đối chiếu khi audit.
- **Scalability**: receiver phải quản lý được nhiều sender trong giới hạn tài nguyên firmware.
- **Maintainability**: mã nguồn chia module rõ ràng để nâng cấp cảm biến, mô hình hoặc giao thức.
- **Usability**: giao diện đủ trực quan để vận hành không phụ thuộc serial log.

2.4 Success criteria mức hệ thống

- 100% điểm đo trong phạm vi triển khai được giám sát liên tục.
- Cảnh báo vượt ngưỡng được phát sinh với độ trễ phù hợp mục tiêu vận hành.
- Dữ liệu lịch sử đủ để truy vết theo dòng thời gian cho mọi sự kiện quan trọng.
- Giảm đáng kể thao tác kiểm tra thủ công và sai sót do con người.



Hình 2: Sơ đồ mạng nội bộ cho kịch bản chuỗi lạnh vaccine: receiver vừa phát SoftAP phục vụ vận hành tại chỗ vừa kết nối hạ tầng Wi-Fi, sender gửi telemetry về gateway để hợp nhất và đẩy cloud.

- nhánh Sender: `https://github.com/dangtai111325/IOT_Application/tree/sender`
- nhánh Receiver: `https://github.com/dangtai111325/IOT_Application/tree/receiver`

Hai liên kết trên được dùng như nguồn tham chiếu gốc cho toàn bộ phần phân tích: luồng khởi tạo, tổ chức module, giao thức TCP nội bộ, cơ chế đồng bộ cấu hình, tích hợp CoreIoT và các giới hạn kỹ thuật runtime. Cách đặt nguồn rõ ràng ngay ở đầu tài liệu giúp người đọc có thể đối chiếu từng luận điểm với mã nguồn tương ứng khi cần.

Phần III

Yêu cầu và định hướng thiết kế

3 Yêu cầu hệ thống và định hướng thiết kế

3.1 Định hướng thiết kế

So với một mẫu (template) chỉ thu thập và chuyển tiếp dữ liệu, giải pháp cần có phạm vi mở rộng rõ: web server trên thiết bị, TinyML tại biên và tích hợp nền tảng cloud (CoreIoT). Khác biệt không chỉ nằm ở một chức năng đơn lẻ mà ở việc tái cấu trúc thành hai lớp thiết bị sender và receiver với luồng điều khiển hai chiều. Cấu trúc đó đưa hệ thống ra khỏi giới hạn một board đơn và bám sát cách triển khai gateway thường gặp trong IoT thực tế.

Điều này đòi hỏi mô tả rõ từng lớp: sender không chỉ đọc dữ liệu mà còn xử lý AI và phục vụ giao diện cục bộ; receiver không chỉ nhận dữ liệu mà còn điều phối mạng cục bộ, quản lý cấu hình và tích hợp cloud. Trình bày theo danh sách file mã nguồn khó thể hiện đầy đủ giá trị thiết kế; vì vậy tài liệu được tổ chức theo trục: *bài toán – kiến trúc – giao thức – module – mã nguồn tiêu biểu – thực nghiệm*.

3.2 Yêu cầu chức năng

Các yêu cầu chức năng chính của hệ thống có thể tóm tắt như sau:

- Mỗi sender phải tạo được dữ liệu nhiệt độ, độ ẩm và vị trí GPS dưới dạng telemetry.
- Sender phải chạy được mô hình TinyML cục bộ và sinh ra `ai_score`, `ai_status`.
- Receiver phải nhận được telemetry từ nhiều sender qua TCP.
- Receiver phải phân biệt sender đã đăng ký và thiết bị lạ dựa trên `device_id`.
- Receiver phải đồng bộ cấu hình xuống sender, hỗ trợ thay đổi threshold và tên hiển thị.
- Cả sender và receiver phải có web UI cục bộ để cấu hình và quan sát trạng thái.
- Receiver phải đẩy được dữ liệu lên CoreIoT qua MQTT gateway và phản hồi RPC `getThreshold`/`setThreshold`.

3.3 Yêu cầu phi chức năng

Ngoài các yêu cầu chức năng, hệ thống còn cần đáp ứng một số yêu cầu phi chức năng quan trọng. Hệ thống phải dễ hiểu để hỗ trợ vận hành, gỡ lỗi và bàn giao kỹ thuật. Mã nguồn cần được chia module hợp lý để dễ bảo trì. Giao thức giữa sender và receiver cần đủ đơn giản để dễ debug, nhưng vẫn phải thể hiện được cấu trúc chuẩn của packet trong hệ thống nhúng. Giao

diện web phải đủ trực quan để người dùng không cần theo dõi serial log liên tục. Cuối cùng, thiết kế phải có khả năng mở rộng để trong tương lai có thể tăng số sender hoặc thay cảm biến giả lập bằng cảm biến thật. Trong cấu hình hiện tại, receiver được thiết kế để quản lý tối đa 10 sender theo đúng giới hạn tài nguyên của firmware.

3.4 Lý do chọn kiến trúc nhiều sender – một receiver

Kiến trúc nhiều sender – một receiver được chọn vì vừa đủ gọn để triển khai và kiểm soát trên phần cứng tài nguyên hạn chế, vừa đủ phong phú để bộc lộ các vấn đề của hệ đa node. Cấu hình demo hiện tại dùng hai sender để giảm độ phức tạp khi kiểm thử. Nếu chỉ có một sender, receiver gần như trở thành một điểm chuyển tiếp thụ động. Khi có từ hai sender trở lên, các vấn đề như quản lý danh tính thiết bị, phân rã trạng thái online/offline, xử lý đồng thời nhiều telemetry và đồng bộ cấu hình trở nên có ý nghĩa thật sự. Mô hình này cũng giúp minh họa rõ kiến trúc, luồng gói tin và dashboard trong tài liệu kỹ thuật.

Phần IV

Kiến trúc và thiết kế các khối

4 Kiến trúc tổng thể và phân rã hệ thống

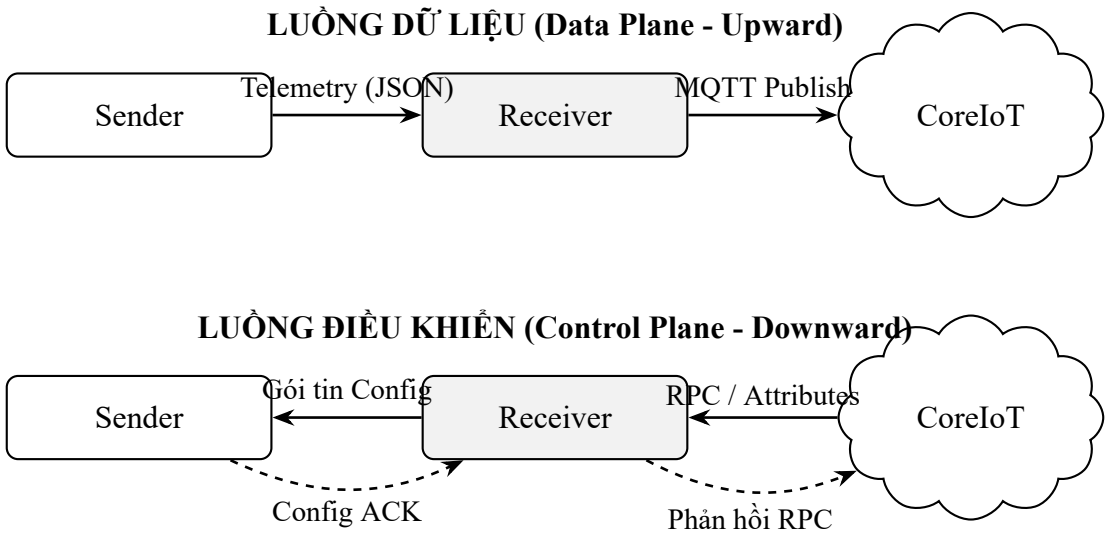
4.1 Bức tranh tổng quát của hệ thống

Hệ thống có thể được nhìn dưới ba lớp. Lớp thứ nhất là *edge sender*, nơi dữ liệu được tạo ra và được xử lý bằng TinyML. Lớp thứ hai là *receiver gateway*, nơi thu thập telemetry, quản lý cấu hình, tập trung web UI và làm đầu mối với cloud. Lớp thứ ba là *CoreIoT*, nơi hiển thị telemetry, lưu trữ thuộc tính và cung cấp RPC từ xa. Tách ba lớp như vậy giúp phân vai hệ thống rõ ràng: sender xử lý cục bộ và đo lường; receiver điều phối và đồng bộ; cloud chịu trách nhiệm trực quan hóa và điều khiển từ xa.

Điểm đáng chú ý là không nên xem receiver chỉ là "board nhận dữ liệu" theo nghĩa hẹp. Trong kiến trúc này, receiver có đầy đủ vai trò của một gateway: nó có bảng trạng thái nhiều sender, có cơ chế persist cấu hình vào LittleFS, có logic retry khi gửi config, có cơ chế publish attribute khi threshold thay đổi, và có xử lý các RPC từ cloud. Đây là yếu tố tạo nên chiều sâu kỹ thuật của thiết kế.

4.2 Mô hình data plane và control plane

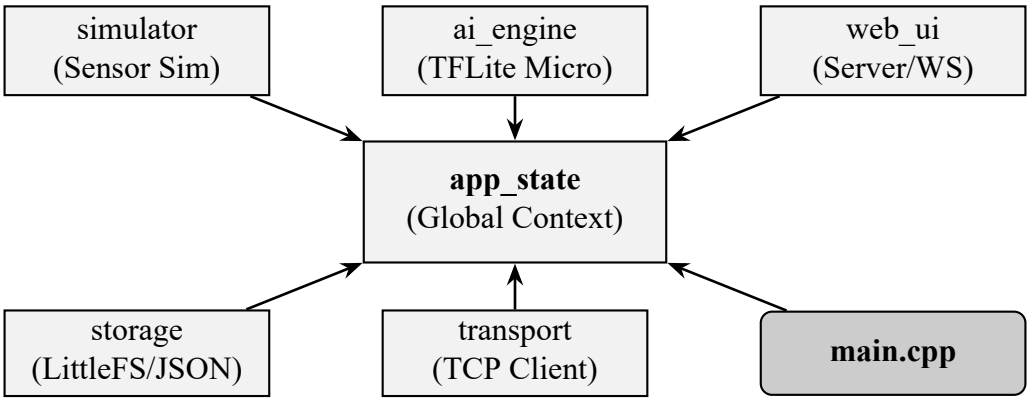
Một cách nhìn khác rất hữu ích khi mô tả hệ thống là tách riêng *data plane* và *control plane*. Data plane là tuyến đường đi lên của dữ liệu cảm biến: sender tạo telemetry, receiver parse và cập nhật `SenderState`, sau đó đẩy tiếp lên CoreIoT. Control plane là tuyến đường đi xuống của cấu hình: thao tác từ Web UI receiver hoặc RPC cloud được receiver chuyển thành packet `config`, sender áp dụng cấu hình và phản hồi `config_ack`. Khi tách như vậy, người đọc sẽ thấy hệ thống không chỉ thu thập dữ liệu mà còn thực hiện vòng phản hồi điều khiển khép kín.



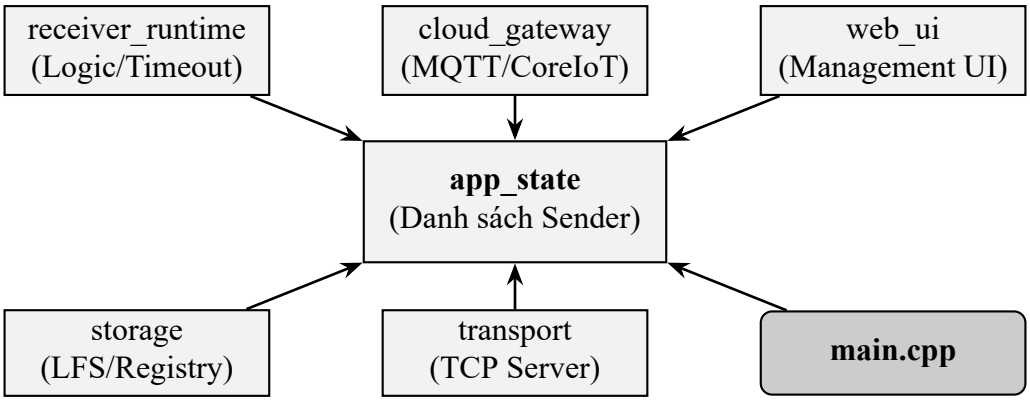
Hình 3: Sơ đồ tách *data plane* và *control plane* giúp nhận diện rõ hai loại luồng trong hệ thống.

4.3 Lợi ích của việc phân rã module

Trong cả sender và receiver, mã nguồn được chia thành các module nhỏ thay vì dồn toàn bộ vào một file. Việc phân rã này mang lại ba lợi ích. Thứ nhất, có thể giải thích từng khối chức năng độc lập mà không rơi vào tình trạng kể lại mã nguồn theo thứ tự dòng. Thứ hai, khi cần thay đổi một chức năng như giao thức mạng hoặc mô hình AI, người phát triển chỉ cần can thiệp vào module liên quan. Thứ ba, việc chia module phản ánh đúng tư duy thiết kế phần mềm nhúng hiện đại: dữ liệu trạng thái ở một nơi, logic mạng ở một nơi, logic UI ở một nơi, còn storage và cloud integration được đóng gói thành các lớp riêng.



Hình 4: Kiến trúc module của sender gồm các khối trạng thái, mô phỏng, AI, transport, lưu trữ và giao diện web.



Hình 5: Kiến trúc module của receiver gồm runtime, transport, cloud gateway, lưu trữ và giao diện web.

4.4 Bảng so sánh phần cứng và phần mềm của hai khối

Tiêu chí	Sender	Receiver
Vai trò chính	Nút biên tạo dữ liệu, chạy TinyML, gửi telemetry	Gateway trung tâm thu nhận, quản lý, đồng bộ cloud
Phần cứng mục tiêu	ESP32 DevKit (32 chân, board `esp32dev`)	ESP32-S3 (YOLO UNO ESP32-S3 trong cấu hình demo)
Chế độ Wi-Fi	AP + STA đồng thời (`WIFI_AP_STA`)	AP + STA đồng thời (`WIFI_AP_STA`)
Điểm truy cập cấu hình	AP cục bộ cho từng sender tại `192.168.7.1`	AP quản trị receiver tại `192.168.7.1`
Giao tiếp nội bộ	TCP client gửi JSON theo dòng lên receiver	TCP server lắng nghe cổng `4100` cho nhiều sender
Định danh thiết bị	`device_id` lấy từ MAC của sender	Quản lý sender theo khóa `device_id = MAC`
Lưu trữ cục bộ	`/sender.json` trên LittleFS	`/receiver.json`, `/senders.json` trên LittleFS
Tích hợp cloud	Không kết nối trực tiếp cloud	Kết nối CoreIoT qua MQTT gateway + RPC

5 Thiết kế chi tiết khối Sender

5.1 Vai trò của sender trong toàn hệ thống

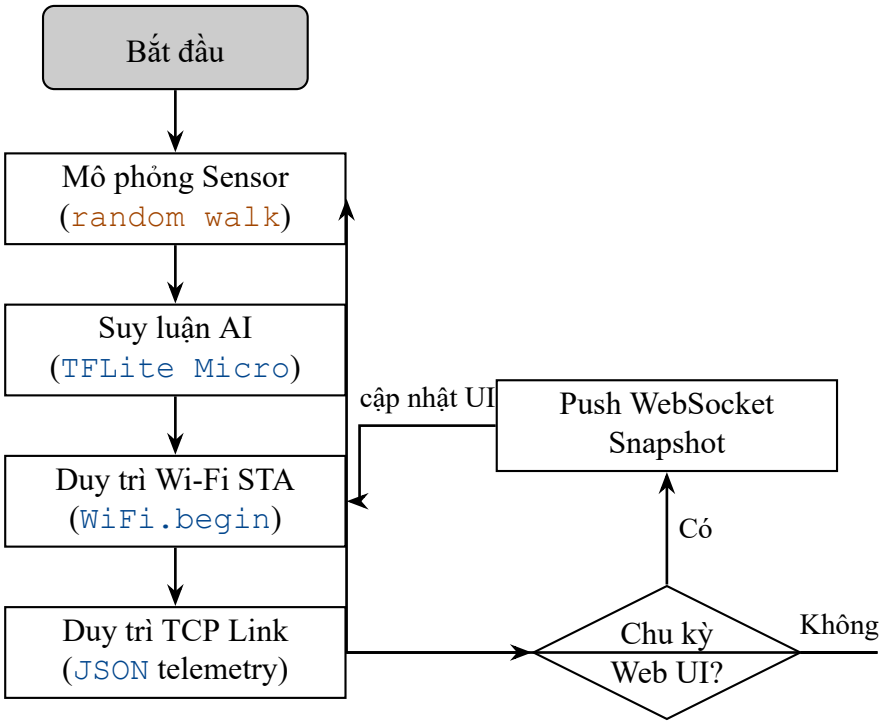
Mỗi sender được thiết kế như một node biên hoàn chỉnh. Nó không đơn thuần lấy dữ liệu rồi gửi đi, mà phải đồng thời thực hiện bốn trách nhiệm: sinh dữ liệu môi trường, chạy mô hình TinyML, duy trì kết nối mạng và cung cấp giao diện cục bộ cho người dùng. Cách thiết kế này

phản ánh đúng tinh thần *edge computing*: xử lý tối thiểu phải diễn ra gần nơi phát sinh dữ liệu để giảm độ trễ và giảm phụ thuộc cloud.

Trong triển khai hiện tại, sender sử dụng dữ liệu giả lập thay cho cảm biến thật. Tuy nhiên, từ góc nhìn kiến trúc, điều này không làm thay đổi vai trò của sender. Module `simulator` chỉ thay thế lớp driver cảm biến; các lớp AI, transport, storage và web UI vẫn vận hành y như khi có cảm biến thật. Nhờ đó có thể tập trung mô tả luồng kỹ thuật cốt lõi mà không phụ thuộc vào một phần cứng cụ thể.

5.2 Luồng khởi tạo sender

Khi khởi động, sender phải hoàn tất một chuỗi bước có tính phụ thuộc lẫn nhau. Trước hết, thiết bị khởi tạo serial để phục vụ debug. Sau đó LittleFS được mount để có thể đọc file cấu hình lưu từ những lần chạy trước. Tiếp theo, sender chuyển Wi-Fi sang chế độ `WIFI_AP_STA`, vì nó vừa cần phát AP cho cấu hình cục bộ, vừa phải kết nối lên mạng ngoài dưới vai trò STA. Sau khi lấy được MAC và sinh `deviceId`, sender tạo SSID local của riêng mình. Từ đây hệ thống lần lượt khởi tạo SoftAP, cố kết nối Wi-Fi ngoài, nạp mô hình TinyML và cuối cùng mở web server cục bộ.



Hình 6: Flowchart phần khởi tạo và vòng lặp sender. Sơ đồ cho thấy sender liên tục mô phỏng cảm biến, chạy AI và duy trì mạng.

5.3 Mô phỏng dữ liệu sensor

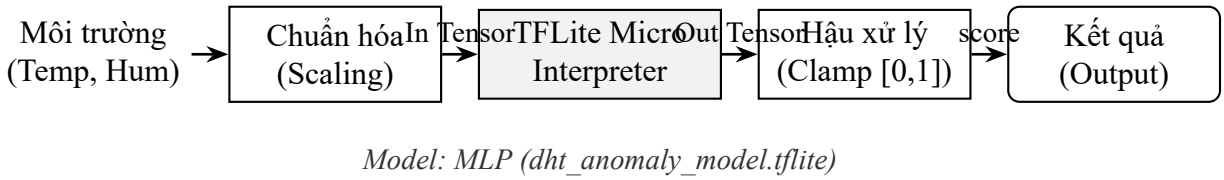
Dữ liệu nhiệt độ và độ ẩm trên sender được mô phỏng theo kiểu *random walk có chặn biên*. Nghĩa là ở mỗi chu kỳ, giá trị mới được hình thành bằng cách cộng thêm một nhiễu ngẫu nhiên

nhỏ vào giá trị cũ, sau đó chặn lại trong miền hợp lý. Cách mô phỏng này tốt hơn việc rút ngẫu nhiên độc lập mỗi lần, vì dữ liệu tạo ra có tính liên tục theo thời gian và nhìn giống dữ liệu môi trường thật hơn. GPS cũng được cập nhật theo cơ chế dịch chuyển nhẹ quanh một vị trí gốc, tạo ra quỹ đạo nhỏ đủ để minh họa dữ liệu vị trí.

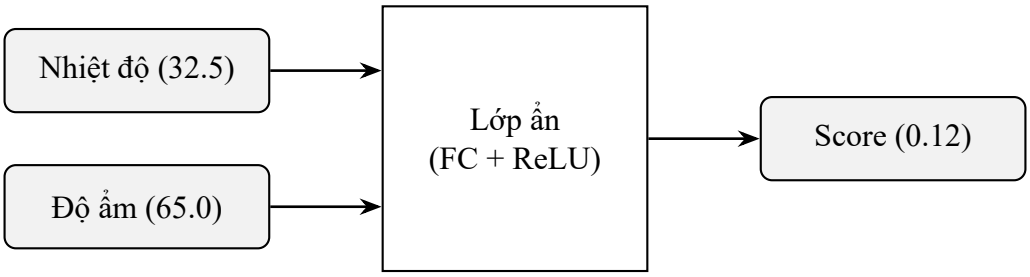
Về mặt trình bày kỹ thuật, cơ chế này đáng được nêu rõ vì nó giải thích tại sao dashboard của sender và receiver hiển thị dữ liệu biến thiên theo thời gian mà vẫn không nhảy loạn hoàn toàn. Dù đây chỉ là dữ liệu giả lập, cách tạo dữ liệu vẫn có dụng ý kỹ thuật để mô hình AI và giao diện trực quan hóa trông có ý nghĩa.

5.4 TinyML inference ở edge

Mô hình TinyML của sender nhận hai đầu vào là nhiệt độ và độ ẩm. Hai giá trị này được đưa vào tensor input, sau đó `MicroInterpreter` của TensorFlow Lite Micro thực hiện suy luận và tạo ra một giá trị `ai_score`. Điểm số này được chặn về khoảng $[0, 1]$ để bảo đảm tính ổn định khi hiển thị và so sánh. Nếu `ai_score` lớn hơn hoặc bằng `threshold` hiện tại, sender gắn nhãn `ANOMALY`; ngược lại, trạng thái là `NORMAL`. Vì sender là nơi tự thực hiện inference, receiver không cần tính lại trạng thái bất thường mà chỉ đóng vai trò tiếp nhận và phân phối thông tin.



Hình 7: Pipeline TinyML trên sender: từ dữ liệu mô phỏng sang tensor input, qua bộ thông dịch TFLite Micro, rồi trả ra điểm bất thường và nhãn trạng thái.



Hình 8: Mô hình logic của TFLite được sử dụng ở mức báo cáo: hai đầu vào đặc trưng và một đầu ra điểm bất thường.

5.5 Lớp mạng và sender transport

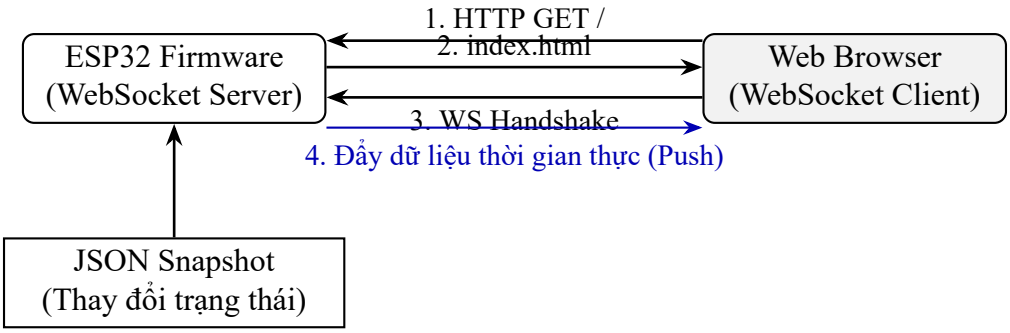
Sender transport là nơi hội tụ ba vai trò quan trọng: phát SoftAP, duy trì kết nối Wi-Fi STA và duy trì TCP client tới receiver. SoftAP cho phép người dùng truy cập giao diện web cục bộ ngay cả khi sender chưa kết nối được Wi-Fi ngoài. Trong khi đó, lớp STA cho phép sender tham gia

vào mạng chung với receiver và CoreIoT. Ở trên hai lớp Wi-Fi đó, sender mở một TCP client tới receiver. Đây là lựa chọn có chủ ý: với số lượng packet không lớn và cần sự đơn giản trong xử lý, dùng TCP với packet JSON theo dòng giúp dễ debug hơn nhiều so với việc tự thiết kế cơ chế truyền tin có xác nhận trên UDP.

Một chi tiết thiết kế khá đặc biệt là sender có thể tự suy ra địa chỉ receiver theo host `.100` nếu đang ở mạng `/24` và người dùng chưa cấu hình địa chỉ receiver bằng tay. Quy tắc này được cài đặt để giảm chi phí cấu hình trong môi trường thử nghiệm. Nếu người dùng nhập IP receiver thủ công thì giá trị này sẽ được ưu tiên sử dụng. Cách tổ chức này vừa giữ được tính tự động khi demo, vừa không làm mất khả năng can thiệp thủ công khi triển khai thực tế.

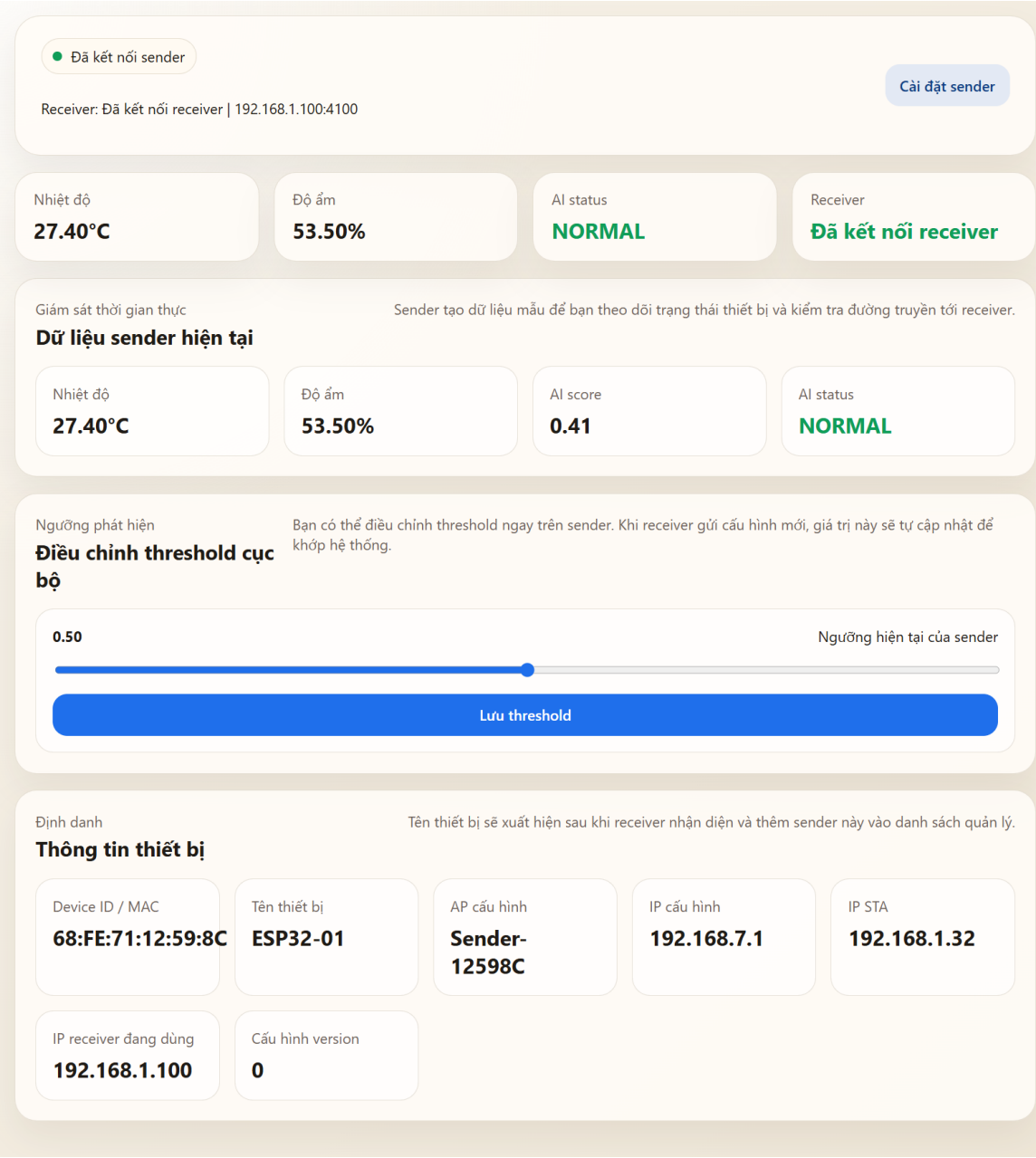
5.6 Giao diện web cục bộ của sender

Sender có web UI riêng để người dùng quan sát trạng thái AI, mạng, cấu hình receiver và điều chỉnh threshold cục bộ. Giao diện này hoạt động nhờ web server phục vụ file tĩnh từ LittleFS và WebSocket `/ws` để đẩy snapshot thời gian thực. Cơ chế WebSocket giúp người dùng không phải refresh trang thủ công mà vẫn thấy được trạng thái mới sau mỗi chu kỳ. Về mặt thiết kế, đây là bước tiến đáng kể so với kiểu hệ nhúng chỉ dựa vào serial monitor.

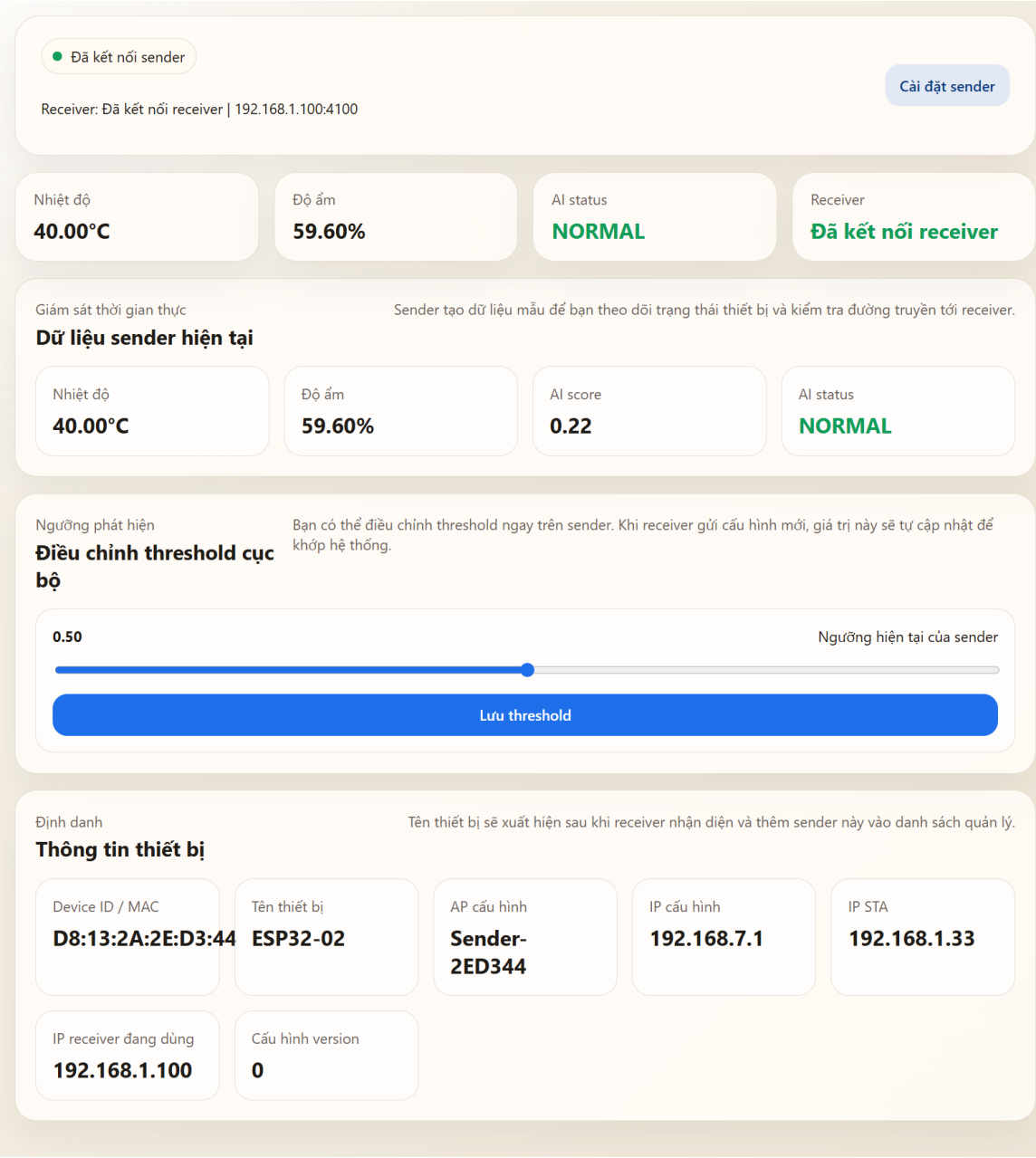


Hình 9: Cơ chế WebSocket snapshot của sender và receiver, cho phép web UI nhận trạng thái thời gian thực mà không cần tải lại trang.

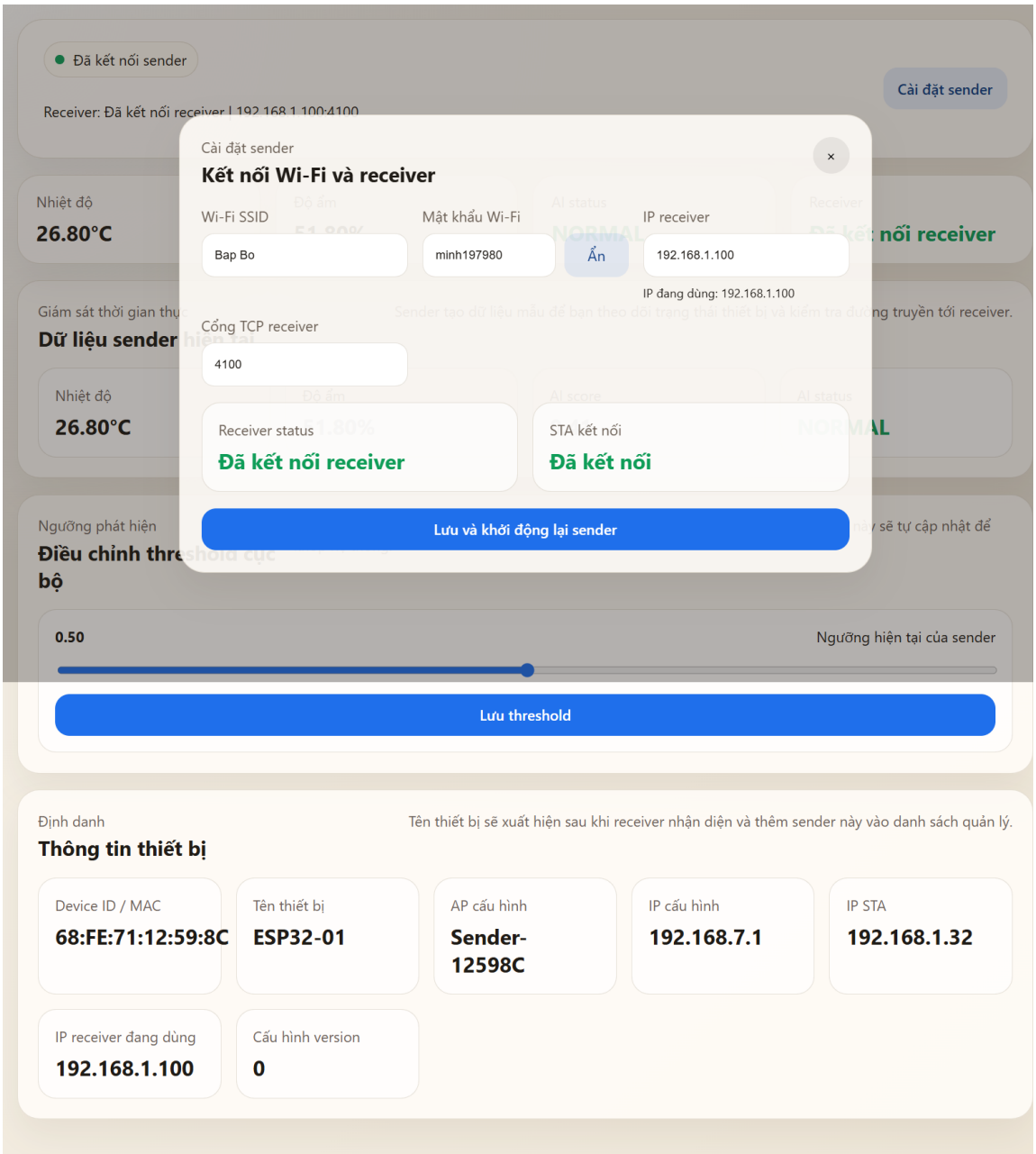
Hình dưới đây minh họa giao diện web của hai sender trong thực nghiệm, cả hai đều kết nối thành công tới receiver và có trạng thái AI bình thường trong điều kiện vận hành thông thường.



Hình 10: Giao diện web sender ESP32-01 trong thực nghiệm.



Hình 11: Giao diện web sender ESP32-02 trong thực nghiệm.



Hình 12: Dialog cài đặt Wi-Fi và receiver của sender.

6 Thiết kế chi tiết khối Receiver

6.1 Receiver như một gateway thực thụ

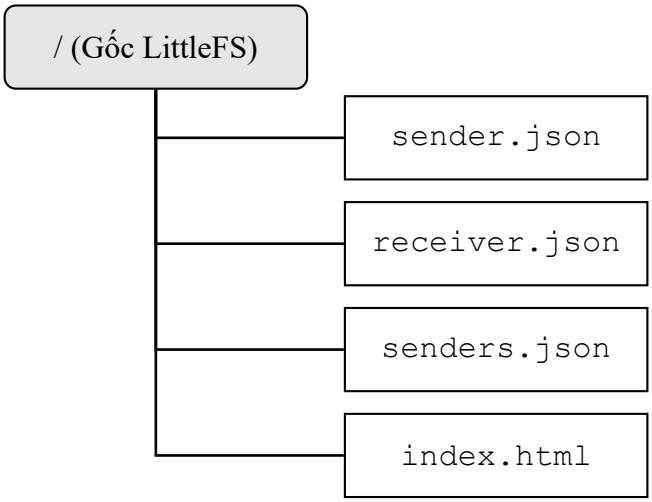
Receiver là khối quan trọng nhất của hệ thống. Nó không chỉ gom dữ liệu lại, mà còn nắm giữ toàn bộ logic điều phối của hệ thống. Receiver chịu trách nhiệm mở SoftAP quản trị, kết nối Wi-Fi ngoài, mở TCP server, theo dõi nhiều socket sender, lưu danh sách sender đã đăng ký, phân biệt thiết bị lạ, đẩy dữ liệu lên cloud, nhận RPC từ CoreIoT và chuyển các yêu cầu cấu hình tới sender tương ứng. Nói cách khác, receiver là nơi toàn bộ dữ liệu và điều khiển hội tụ.

Từ góc nhìn mô tả kiến trúc, việc nhấn mạnh vai trò gateway là rất quan trọng. Nếu xem receiver như một thiết bị "thu và hiển thị", người đọc sẽ đánh giá thấp cấu trúc thật của hệ thống. Thực tế, các module của receiver cùng nhau tạo thành một phần mềm quản trị node biên thu nhỏ, với đầy đủ các khía cạnh networking, persistence, protocol parsing và cloud integration.

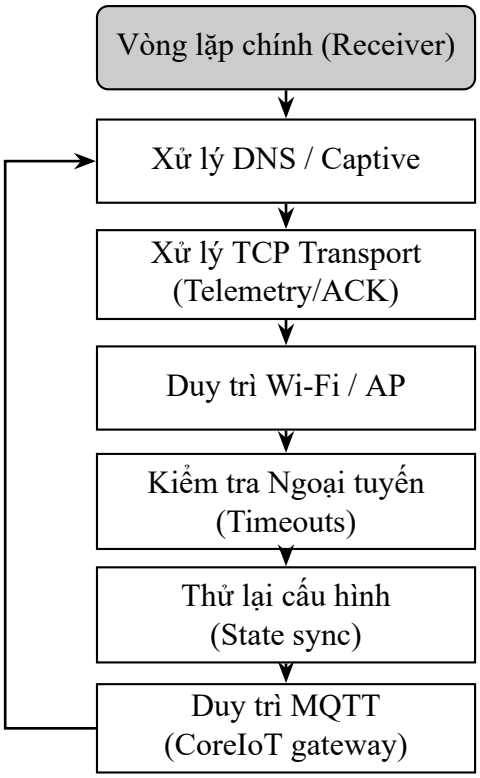
6.2 Luồng khởi tạo receiver

Quá trình khởi tạo receiver bắt đầu bằng việc mount LittleFS, nạp file cấu hình receiver và nạp danh sách sender đã đăng ký từ file `/senders.json`. Sau đó receiver thiết lập đồng thời SoftAP, DNS server và Wi-Fi STA. Việc khởi tạo TCP server được thực hiện riêng thành một bước để hệ thống sẵn sàng nhận sender mới bất cứ lúc nào. Cuối cùng receiver mở web server và WebSocket để chuẩn bị cho giao diện quản trị. Đây là trình tự hợp lý vì receiver phải có state và cấu hình trước khi có thể phục vụ UI hoặc xử lý sender.

Cả sender và receiver đều dùng LittleFS làm lớp lưu trữ bền vững. Tệp `sender.json` và `receiver.json` lưu cấu hình thiết bị qua các lần khởi động; `senders.json` lưu danh sách sender đã đăng ký ở receiver; `index.html` là giao diện web được phục vụ trực tiếp từ flash. Hình 13 minh họa cây thư mục đơn giản này.



Hình 13: Cấu trúc lưu trữ LittleFS dùng chung cho cả sender và receiver: tệp cấu hình, danh sách sender đã đăng ký và giao diện web tĩnh.

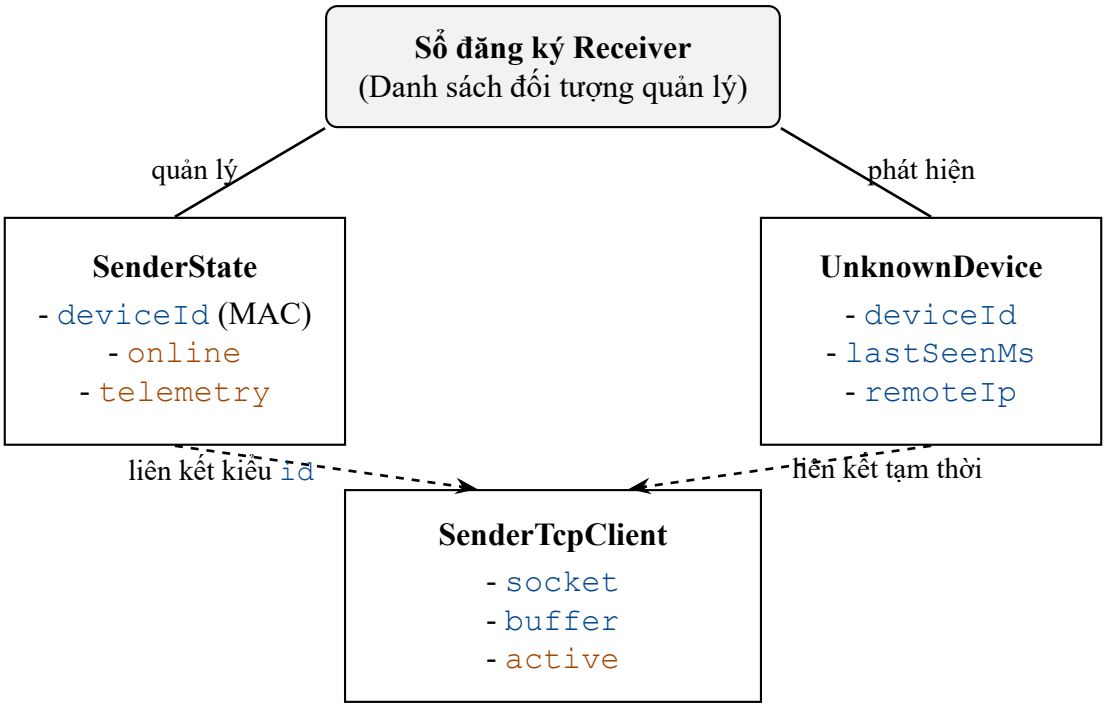


Hình 14: Flowchart phần khởi tạo và vòng lặp receiver. Sơ đồ cho thấy receiver đồng thời xử lý DNS, TCP, Wi-Fi, MQTT và snapshot cho UI.

6.3 Mô hình dữ liệu trung tâm của receiver

Receiver dùng bốn cấu trúc dữ liệu quan trọng. `ReceiverConfig` giữ thông tin Wi-Fi, server MQTT và token gateway. `SenderState` lưu toàn bộ trạng thái của sender đã đăng ký: online/offline, telemetry, threshold, cấu hình chờ đồng bộ, nonce cloud và địa chỉ remote. `UnknownDevice` dùng để theo dõi những sender đã gửi telemetry nhưng chưa nằm trong danh sách đăng ký. Cuối cùng `SenderTcpClient` biểu diễn từng socket TCP còn sống, kèm buffer để ghép các dòng JSON chưa hoàn tất.

Mô hình dữ liệu này là một điểm nổi bật vì nó cho thấy receiver không quản lý sender theo kiểu mảng giá trị đơn giản, mà phân chia rất rõ các lớp thông tin: cấu hình gateway, sender hợp lệ, thiết bị lạ và socket cấp thấp. Điều đó giúp phần trình bày mạch lạc hơn khi giải thích luồng hoạt động.

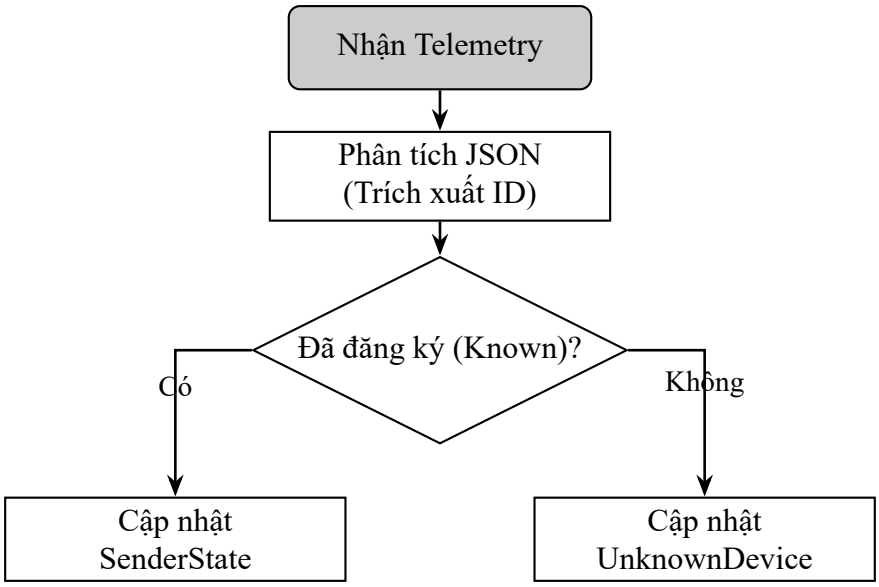


Hình 15: Mô hình dữ liệu trung tâm của receiver gồm cấu hình gateway, trạng thái sender đã đăng ký, thiết bị lạ và các kết nối TCP đang sống.

6.4 Known sender và unknown device

Khi receiver nhận telemetry, việc đầu tiên không phải là đẩy dữ liệu lên cloud mà là kiểm tra `device_id`. Trong hệ thống này, `device_id` được gắn trực tiếp bằng địa chỉ MAC của sender để có khóa định danh ổn định xuyên suốt. Nếu `device_id` đã có trong bảng sender đăng ký thì telemetry được gắn vào `SenderState`. Nếu không, receiver chỉ đánh dấu thiết bị đó là `UnknownDevice`. Cách làm này có hai ý nghĩa. Thứ nhất, nó ngăn hệ thống tự động tin tưởng mọi thiết bị lạ xuất hiện trong mạng. Thứ hai, nó cho phép giao diện web hiển thị rõ rằng hiện có thiết bị chưa được quản lý chính thức, từ đó người dùng có thể chủ động đăng ký hoặc bỏ qua.

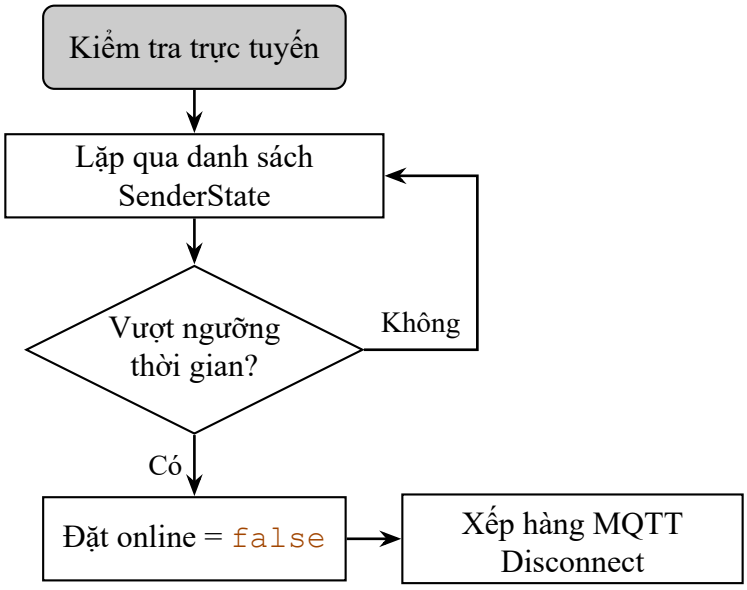
Đối với thiết bị lạ, receiver áp dụng nguyên tắc xử lý nhẹ: chỉ lưu các thông tin nhận diện và nhịp xuất hiện (`MAC/IP/lastSeen/seenCount`), không kích hoạt đầy đủ pipeline cloud telemetry. Quy tắc này giúp tiết kiệm tài nguyên và giữ hệ thống ổn định khi có lưu lượng không mong muốn.



Hình 16: Flowchart phân loại telemetry theo mã thiết bị: sender đã đăng ký được cập nhật vào bảng trạng thái, còn sender chưa đăng ký được đưa vào danh sách thiết bị lạ.

6.5 Cơ chế online/offline

Receiver không phụ thuộc vào sự kiện ngắt kết nối TCP duy nhất để xác định sender offline. Thay vào đó, hệ thống dùng chiến lược timeout dựa trên `lastSeenMs`. Mỗi khi nhận telemetry hoặc ACK, receiver cập nhật `lastSeenMs`. Nếu quá `OFFLINE_TIMEOUT_MS` mà không thấy dữ liệu mới, sender được chuyển sang trạng thái offline. Với sender đã từng kết nối cloud, receiver còn tạo `pendingDisconnect` để gửi sự kiện disconnect lên CoreIoT. Cơ chế này phản ánh cách hệ thống phân tán thường quản lý *liveness*: trạng thái online là kết quả của heartbeat gần nhất, không phải một cờ tĩnh.

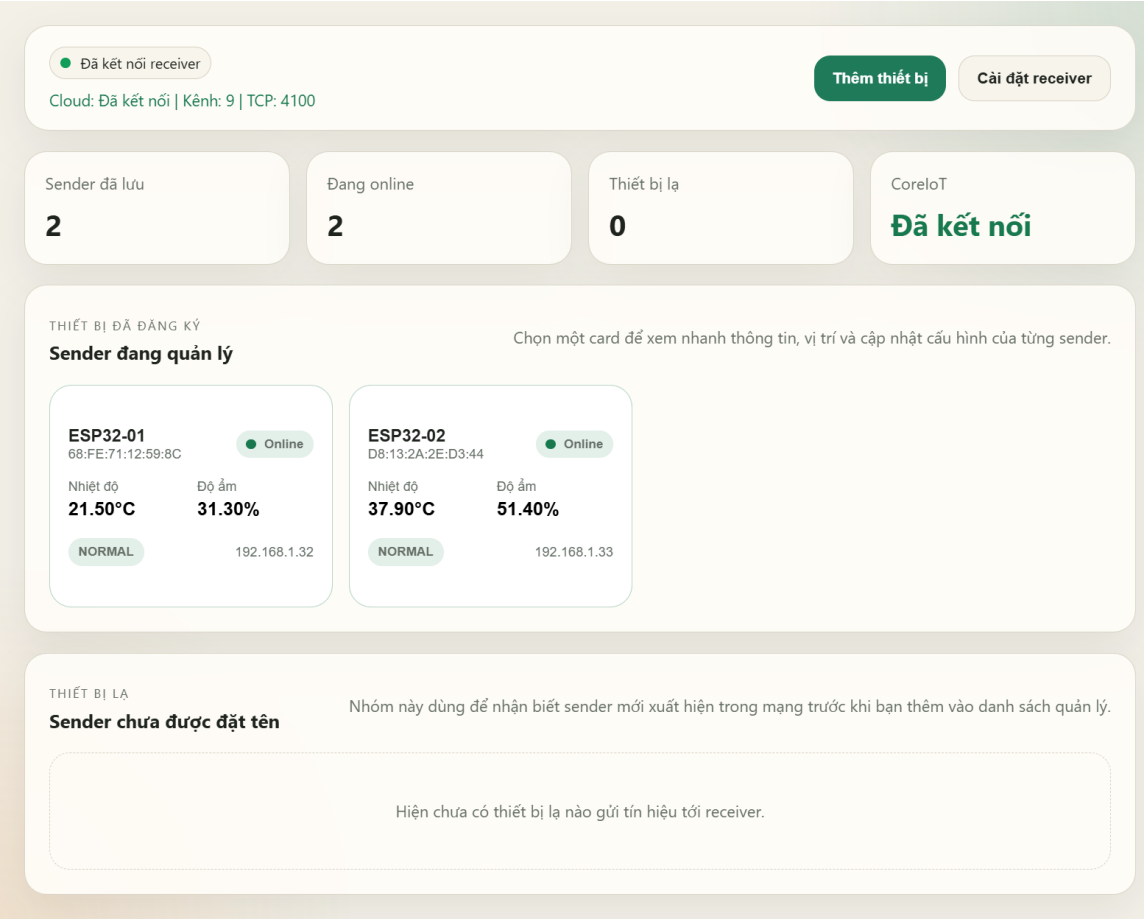


Hình 17: Flowchart chuyển trạng thái online/offline của sender trên receiver dựa vào timeout.

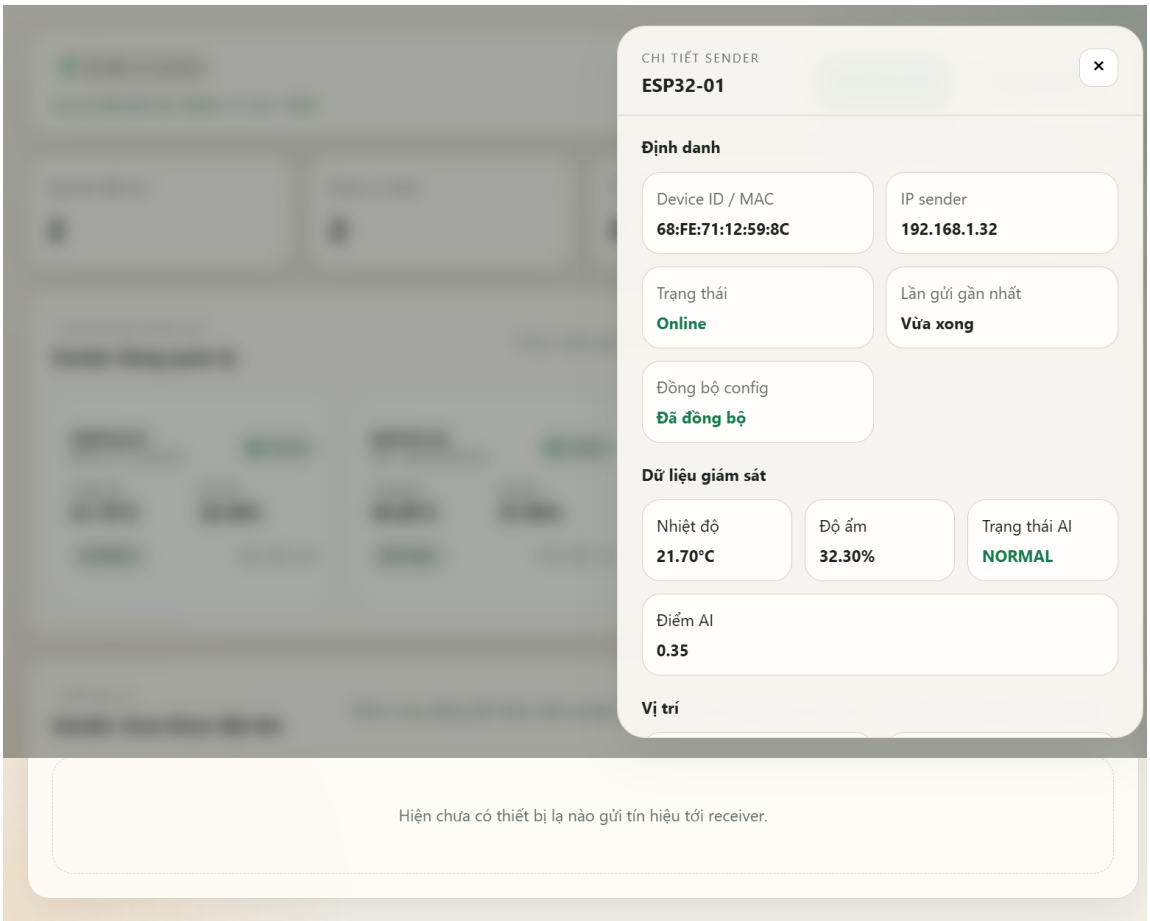
6.6 *Web UI quản trị của receiver*

Giao diện web của receiver có hai vai trò. Vai trò thứ nhất là quan sát trạng thái tập trung của toàn hệ thống: thông tin receiver, cloud status, danh sách sender, unknown devices và đồng bộ cấu hình. Vai trò thứ hai là cho phép quản trị: thêm sender, gán tên, đặt threshold mong muốn, xóa sender, lưu cấu hình Wi-Fi hoặc CoreIoT. Việc gom các chức năng này vào receiver là hợp lý vì receiver là node duy nhất nhìn thấy đầy đủ cả hệ thống.

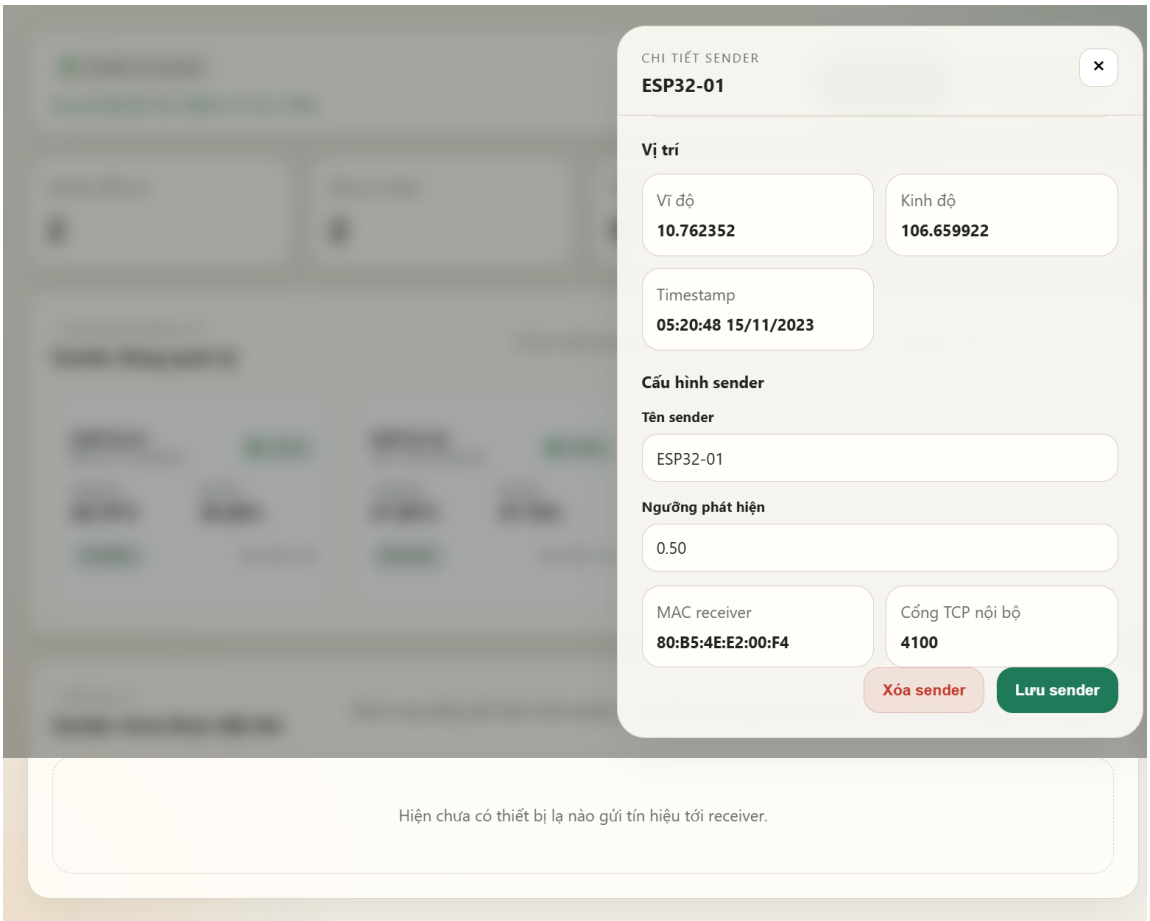
Các hình dưới minh họa giao diện receiver trong thực nghiệm với hai sender đang hoạt động đồng thời: tổng quan hệ thống, chi tiết từng sender và cài đặt kết nối CoreIoT.



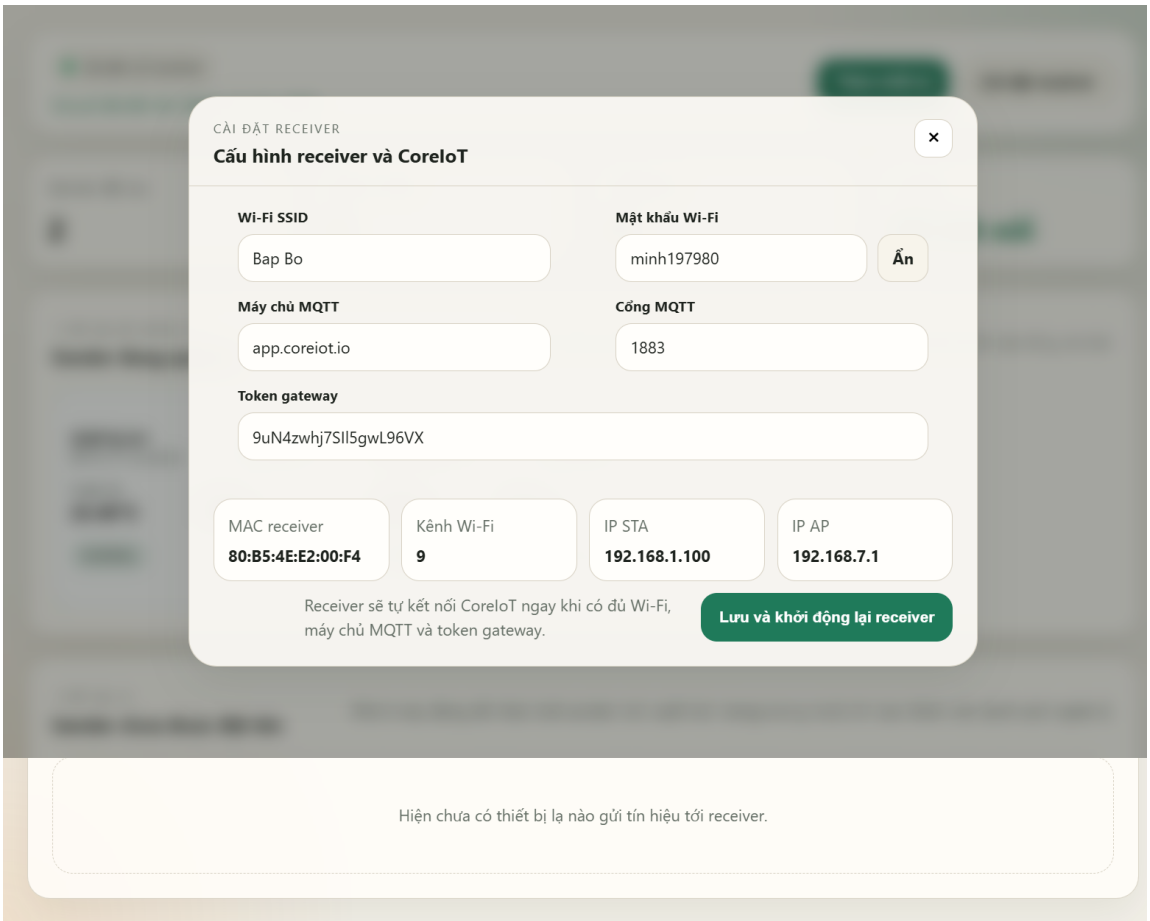
Hình 18: Giao diện receiver: tổng quan 2 sender online và trạng thái CoreIoT.



Hình 19: Chi tiết sender ESP32-01 trên giao diện receiver — thông tin cơ bản.



Hình 20: Chi tiết sender ESP32-01 — vị trí, cấu hình và kết nối TCP.



Hình 21: Dialog cài đặt receiver và CoreIoT.

Phần V

Giao thức, luồng điều khiển và tích hợp cloud

7 Thiết kế giao thức truyền thông

7.1 Lý do chọn TCP nội bộ

Một quyết định thiết kế đáng phân tích là việc dùng TCP cho đường sender – receiver. Trong những bài demo nhỏ, người ta thường dùng giao thức càng đơn giản càng tốt, hoặc thậm chí gửi broadcast không có xác nhận. Tuy nhiên, trong hệ thống này cần một đường truyền ổn định, dễ debug, và thuận tiện để triển khai cơ chế cấu hình hai chiều. Dùng TCP mang lại lợi ích ở chỗ gói tin được đảm bảo thứ tự, giảm gánh nặng tự xử lý truyền lại ở tầng ứng dụng. Thêm vào đó, khi packet ở dạng JSON theo dòng, việc đọc log và phân tích sai lệch khi debug trở nên rất trực quan.

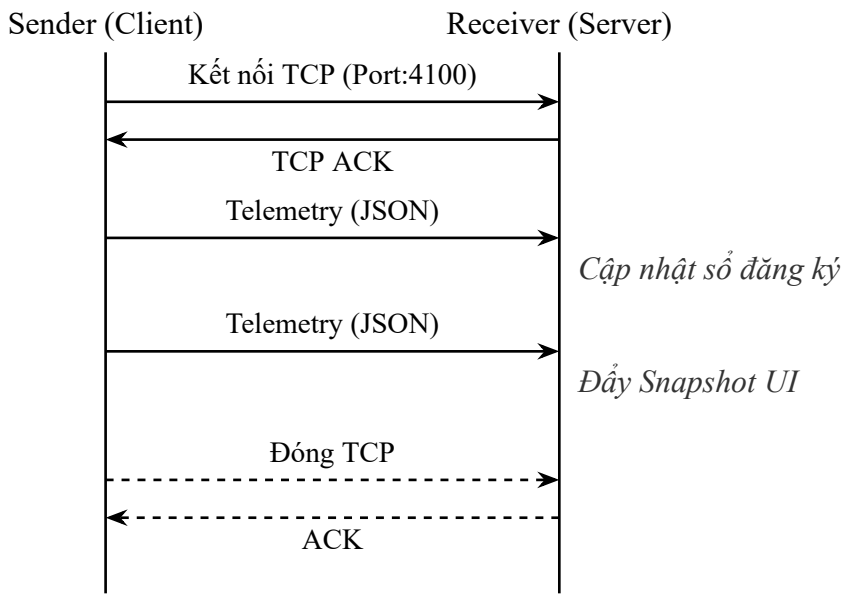
TCP nội bộ cũng phù hợp với receiver đóng vai trò server và sender đóng vai trò client. Kiến trúc này giúp sender chủ động kết nối về receiver sau khi đã có Wi-Fi STA, trong khi receiver luôn lắng nghe ở một port cố định. Nhờ đó, khi sender rơi mạng rồi quay lại, chính sender là nơi tái tạo kết nối mà không cần receiver quét lại mạng.

7.2 Định dạng packet TCP

Trên đường TCP, hệ thống sử dụng ba loại packet chính: `telemetry`, `config` và `config_ack`. Packet `telemetry` là gói sender gửi lên receiver theo chu kỳ. Packet `config` do receiver gửi xuống sender khi cần thay đổi threshold hoặc metadata. Packet `config_ack` do sender trả lại sau khi áp dụng hoặc từ chối cấu hình. Thiết kế chỉ dùng ba loại gói đã đủ để xây dựng một vòng điều khiển khép kín đơn giản nhưng rõ ràng. Toàn bộ các gói này đi qua TCP server của receiver trên cổng `4100`.

Loại Gói	Hướng	Các Trường Dữ liệu Chính
`telemetry`	S → R	`id, temp, hum, ai_score, ai_status, …`
`config`	R → S	`id, threshold, name, version, …`
`config_ack`	S → R	`id, success, applied_threshold, …`

Hình 22: Bảng tổng hợp ba loại message TCP và các trường dữ liệu chính được trao đổi giữa sender và receiver.



Hình 23: Sequence diagram mô tả nhiều sender gửi telemetry lên receiver, receiver cập nhật trạng thái và tiếp tục publish lên cloud.

7.3 Chi tiết packet telemetry

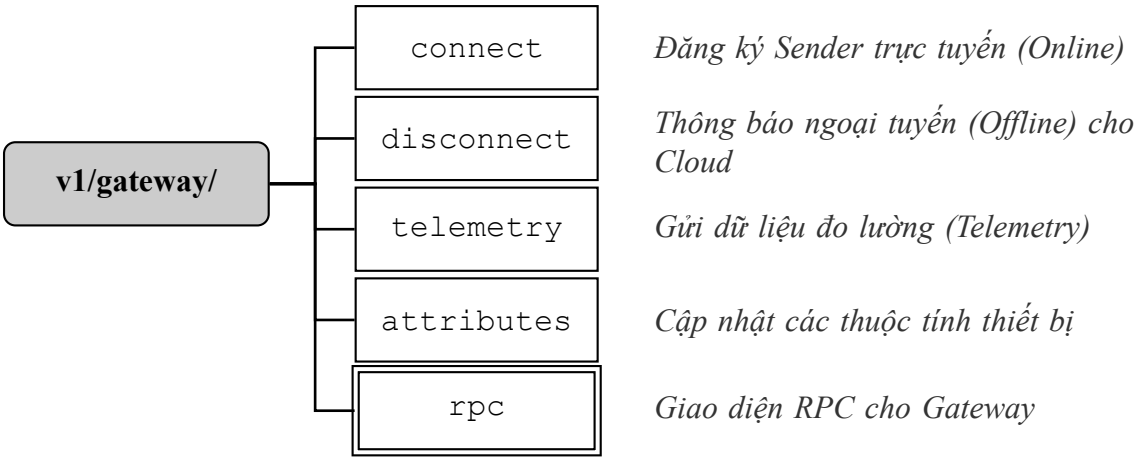
Packet `telemetry` chứa `device_id`, nhiệt độ, độ ẩm, threshold hiện tại trên sender, `ai_score`, `ai_status`, tọa độ GPS và timestamp. Việc gửi cả threshold hiện tại lên receiver có một ý nghĩa tinh tế: nếu sender thay đổi threshold cục bộ bằng giao diện web của chính nó, receiver vẫn có thể phát hiện sự thay đổi này và học lại giá trị mới khi không có phiên cấu hình nào đang chờ. Điều này làm cho hệ thống linh hoạt hơn thay vì bắt buộc mọi thay đổi đều phải đi qua gateway.

7.4 Chi tiết packet config và ACK

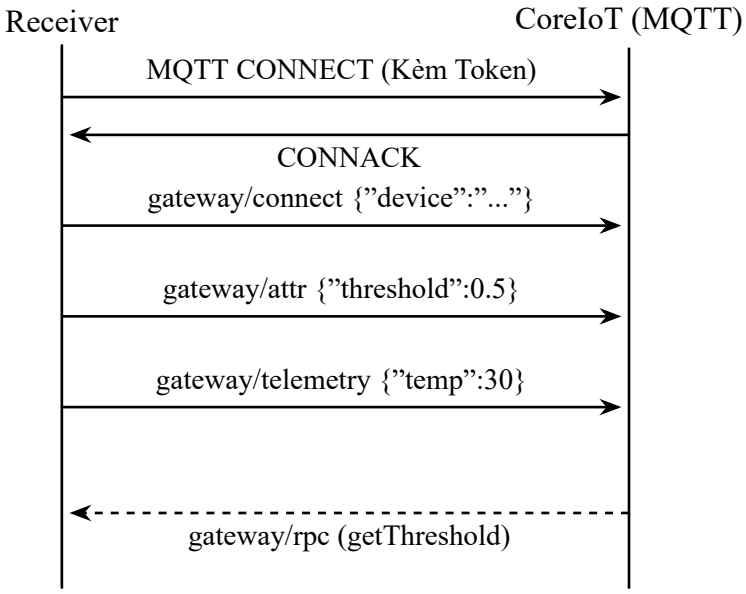
Packet `config` do receiver sinh ra từ `SenderState` gồm `device_id`, tên sender, threshold mong muốn, địa chỉ IP/port của receiver và `configVersion`. Sender chỉ áp dụng gói nếu `device_id` trùng với chính nó. Sau khi lưu lại cấu hình, sender trả về `config_ack` với `success` và `appliedThreshold`. Nhờ ACK, receiver không phải suy đoán việc cấu hình đã thành công hay chưa. Đây là lý do hệ thống có thể hỗ trợ retry một cách minh bạch.

7.5 Giao thức MQTT với CoreIoT

Với CoreIoT, receiver dùng mô hình MQTT gateway. Điều này có nghĩa receiver không mở một MQTT session riêng cho từng sender mà dùng một client duy nhất rồi đại diện cho nhiều sender qua các topic gateway. Cách làm này rất đúng với vai trò gateway: sender không cần biết gì về token cloud, không phải tự kết nối Internet trực tiếp với broker, và toàn bộ kiểm soát cloud được tập trung ở receiver. Từ đó, việc thay đổi cấu hình cloud chỉ cần thực hiện ở một nơi.



Hình 24: Cấu trúc phân cấp topic MQTT Gateway dùng cho tích hợp CoreIoT.



Hình 25: Sequence diagram mô tả receiver đại diện sender khi gửi connect, telemetry, attribute và xử lý RPC qua MQTT gateway.

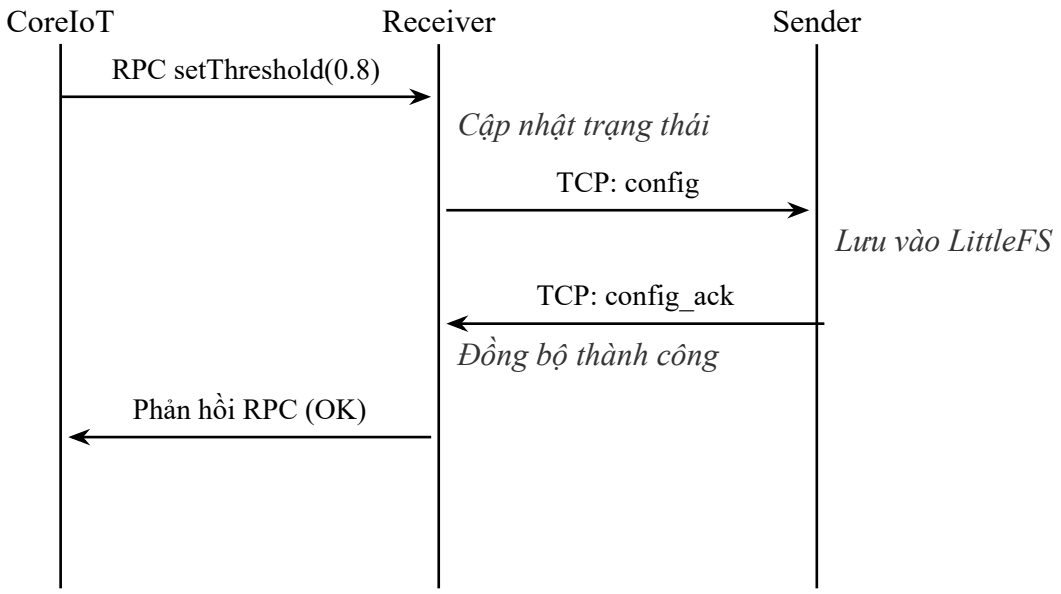
8 Thiết kế luồng điều khiển cấu hình và RPC

8.1 Tại sao cần đồng bộ cấu hình hai chiều

Trong hệ thống nhiều node, việc cấu hình nằm rải rác ở từng sender sẽ nhanh chóng khó quản lý. Nếu người vận hành thay đổi threshold trên một sender qua web local, thay sender khác qua cloud, rồi muốn kiểm tra trạng thái cuối cùng ở receiver, hệ thống cần một mô hình đồng bộ đủ rõ để tránh mâu thuẫn. Đó là lý do receiver duy trì đồng thời `reportedThreshold`, `desiredThreshold`, `pendingConfig`, `awaitingAck`, `configVersion` và `ackedConfigVersion`. Bộ biến này là xương sống của control plane.

8.2 Luồng đổi threshold từ receiver UI

Khi người dùng thao tác trên receiver UI để thay đổi threshold của một sender, receiver trước tiên kiểm tra tính hợp lệ của `device_id`, tên hiển thị và ngưỡng mới. Sau đó receiver ghi `desiredThreshold`, tăng `configVersion`, bật `pendingConfig` và nếu sender đang online thì lập tức gửi `config`. Sender nhận gói, áp dụng ngưỡng mới, lưu xuống LittleFS rồi trả `config_ack`. Chỉ khi nhận được ACK hợp lệ, receiver mới đánh dấu quá trình đồng bộ hoàn tất. Cơ chế này giúp giao diện receiver phản ánh đúng rằng thay đổi đã thực sự tới được sender, thay vì chỉ là ý định cấu hình.



Hình 26: Sequence diagram mô tả quá trình receiver gửi packet cấu hình xuống sender và nhận lại gói xác nhận. Cùng cơ chế này cũng được dùng khi có RPC từ cloud.

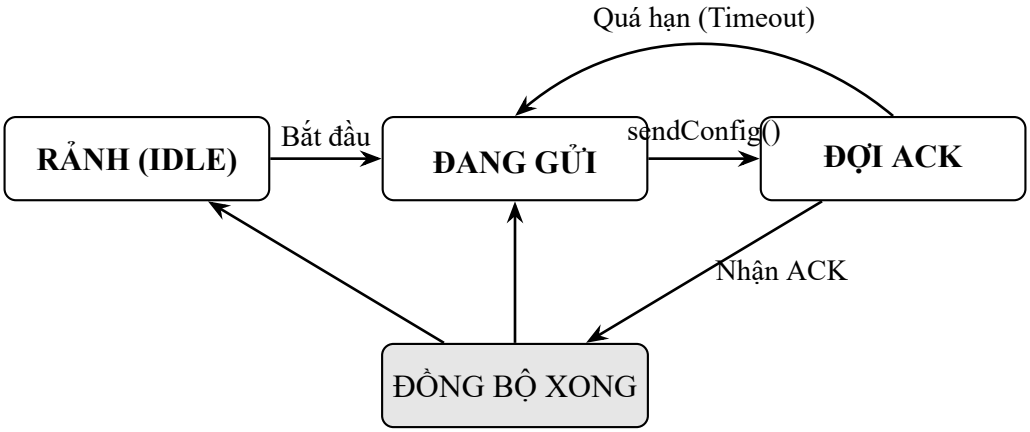
8.3 Luồng đổi threshold từ RPC cloud

Khi CoreIoT gửi RPC `setThreshold`, receiver không cập nhật sender ngay ở cloud mà chuyển đổi RPC đó thành một phiên cấu hình nội bộ. Nghĩa là receiver parse phương thức, tìm sender theo tên, cập nhật `desiredThreshold`, rồi gọi chính cơ chế `sendConfigPacket()` như ở Web UI local. Khi sender ACK thành công, receiver mới gửi RPC response ngược lại cloud. Với cách này, cloud chỉ đóng vai trò nguồn phát lệnh; còn receiver vẫn là nơi duy nhất có quyền điều phối sender. Đây là thiết kế đúng với mô hình gateway và giúp mọi đường điều khiển đều hội tụ về một lớp logic thống nhất.

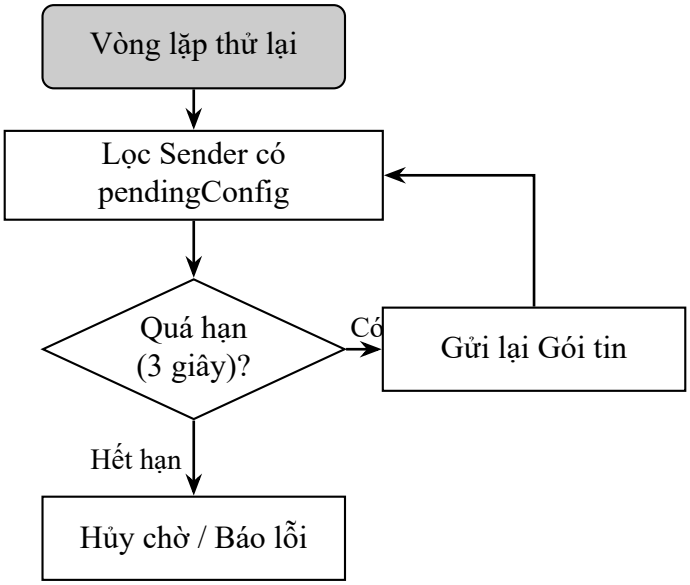
8.4 Retry và timeout

Nếu sender chưa ACK sau một khoảng thời gian định trước, receiver không xem đó là thất bại vĩnh viễn ngay lập tức. Thay vào đó, `processConfigRetries()` sẽ kiểm tra các sender đang có `pendingConfig`. Nếu sender vẫn online nhưng `awaitingAck` đã quá timeout, hệ

thống hạ cờ `awaitingAck` và có thể thử gửi lại. Nếu yêu cầu xuất phát từ RPC cloud, receiver đồng thời tạo phản hồi lỗi phù hợp để platform biết rằng sender không xác nhận kịp thời. Đây là ví dụ rõ ràng cho thấy một hệ thống nhúng nhỏ vẫn có thể tổ chức logic đồng bộ có trạng thái một cách bài bản.



Hình 27: State machine đồng bộ cấu hình giữa receiver và sender.



Hình 28: Flowchart cơ chế retry cấu hình của receiver khi sender chưa ACK trong thời gian cho phép.

9 Tích hợp CoreIoT và vai trò của cloud

9.1 Receiver như một MQTT gateway

Receiver dùng đúng tinh thần *gateway* khi kết nối CoreIoT. Khi một sender online và đã đăng ký, receiver gửi `v1/gateway/connect` để mở phiên thiết bị trên cloud. Ngay sau đó, receiver có thể gửi `v1/gateway/attributes` để đồng bộ metadata như `device_id`, `receiver_ip` và threshold. Tiếp theo, mỗi khi có telemetry mới, receiver

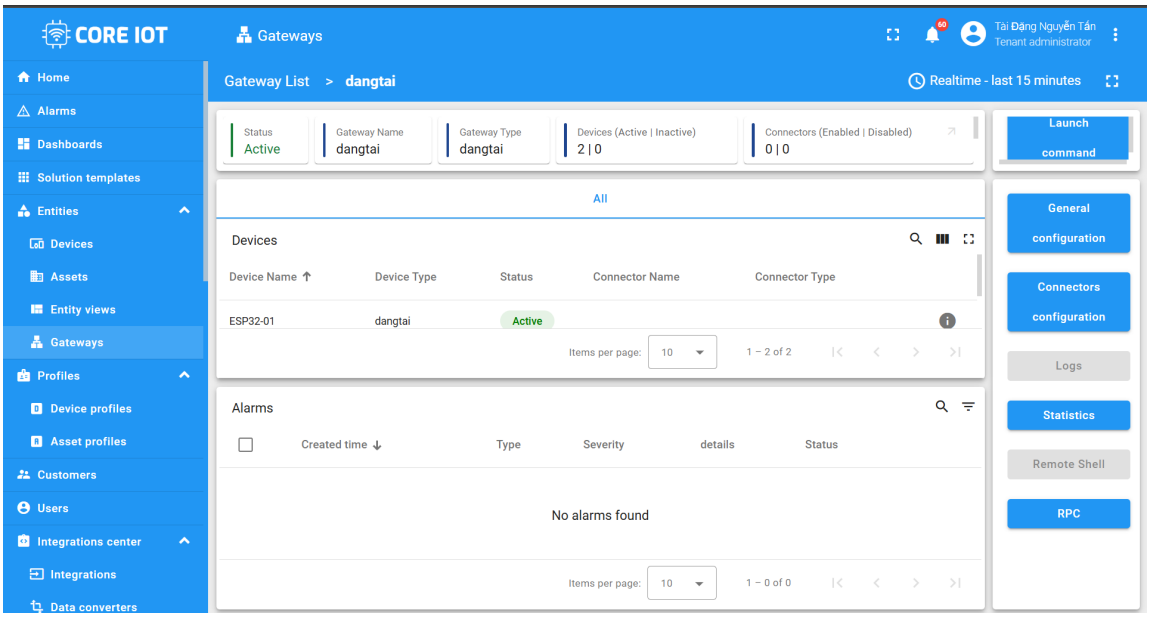
publish dữ liệu lên `v1/gateway/telemetry`. Nếu sender offline hoặc bị xóa, receiver gửi `v1/gateway/disconnect`. Nhờ cơ chế này, cloud nhìn thấy mỗi sender như một thực thể riêng, dù về mặt vật lý chỉ receiver mới là thiết bị trực tiếp mở phiên MQTT.

Việc tách `attributes` và `telemetry` cũng là một quyết định quan trọng. Dữ liệu biến thiên nhanh (nhiệt độ, độ ẩm, ai_score, trạng thái AI, GPS) được gửi theo telemetry; dữ liệu định danh/cấu hình (`device_id`, threshold, thông tin receiver) được gửi theo attributes và chỉ cập nhật khi cần. Cách tách này giúp dashboard dễ đọc hơn và giảm phát sinh dữ liệu lặp không cần thiết.

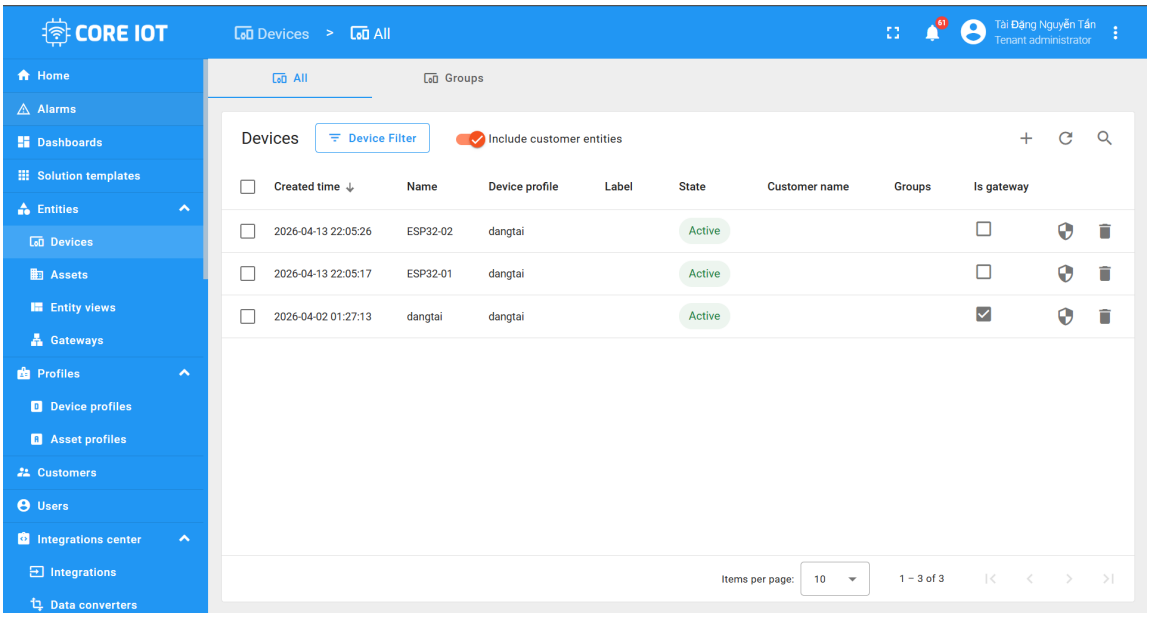
9.2 Giá trị của tầng cloud đối với hệ thống

Trong kiến trúc đang mô tả, cloud không được dùng để thay thế web UI cục bộ mà được xem là tầng giám sát và điều khiển từ xa. Điều này rất quan trọng. Web UI cục bộ giúp cấu hình nhanh trong môi trường thử nghiệm, nhất là khi chưa có Internet hoặc khi cần truy cập AP tại chỗ. Cloud lại có vai trò lịch sử hóa, trực quan hóa tập trung và điều khiển từ xa. Cách tách hai vai trò như vậy giúp kiến trúc cân bằng: local UI phục vụ vận hành tại chỗ, cloud phục vụ quản trị từ xa.

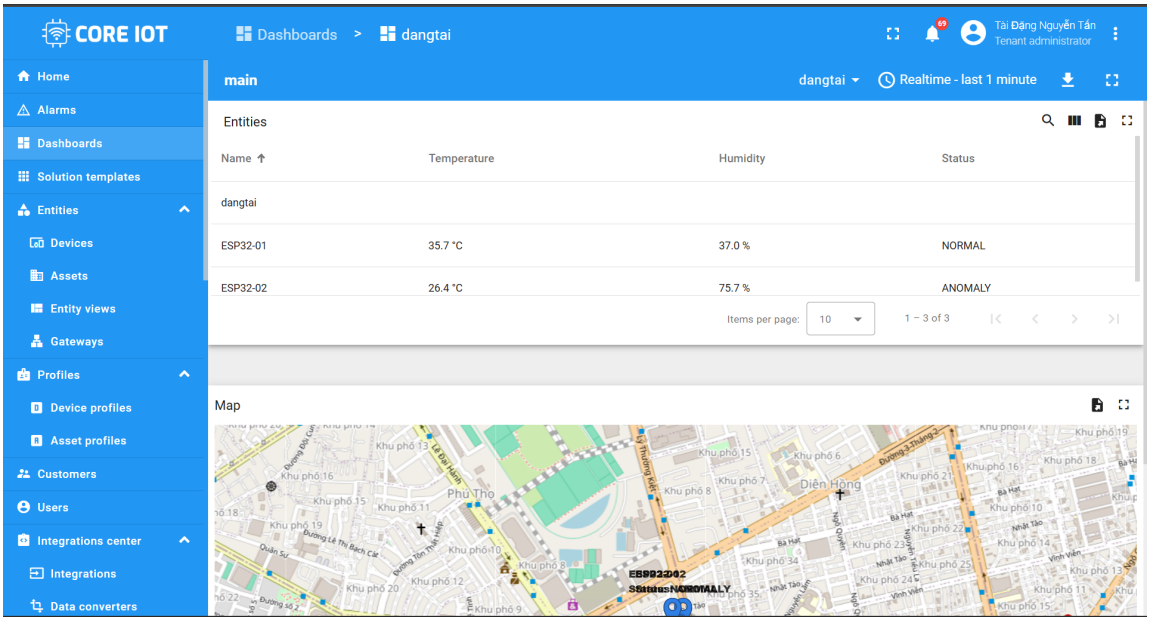
Các hình dưới minh họa kết quả tích hợp receiver với CoreIoT trong thực nghiệm: từ quản lý gateway, danh sách thiết bị, dashboard telemetry đến chi tiết từng sensor và biểu đồ lịch sử.



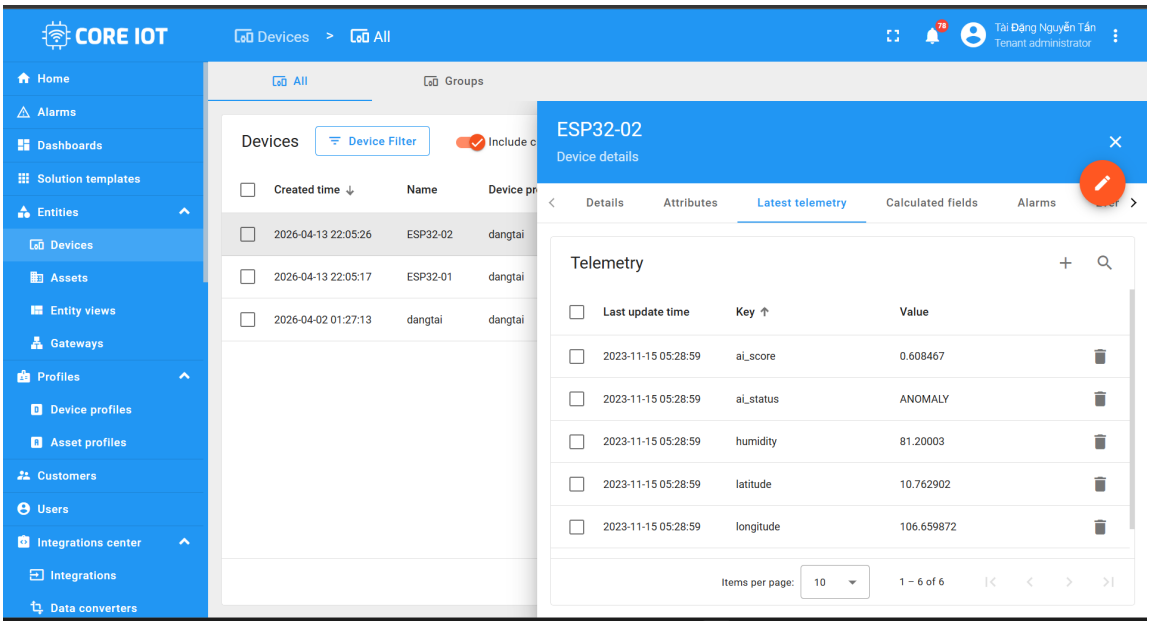
Hình 29: Gateway dangtai trên CoreIoT với 2 thiết bị ESP32 đang hoạt động.



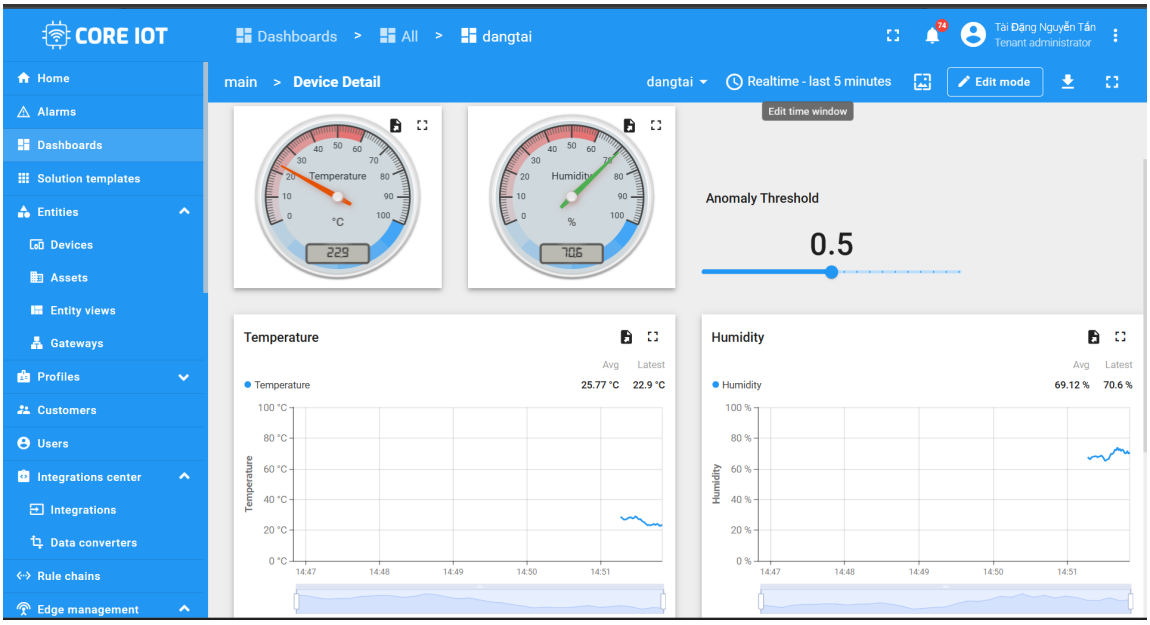
Hình 30: Danh sách thiết bị CoreIoT sau khi receiver đăng ký hai sender.



Hình 31: Dashboard CoreIoT: telemetry và trạng thái AI của hai sender.



Hình 32: Telemetry ESP32-02 trên CoreIoT — ai_score ANOMALY và GPS.



Hình 33: Dashboard chi tiết thiết bị CoreIoT: gauge, biểu đồ lịch sử và threshold.

9.3 Ý nghĩa của việc hỗ trợ RPC

Hỗ trợ RPC `getThreshold/setThreshold` giúp hệ thống tiến gần hơn tới mô hình điều khiển từ xa thực sự. Thay vì cloud chỉ là nơi xem dữ liệu, người vận hành có thể gửi một yêu cầu điều chỉnh threshold từ dashboard. Yêu cầu đó đi qua receiver và được xác nhận bởi sender. Vì có vòng ACK nên RPC phản ánh chính xác trạng thái thực của thiết bị, chứ không chỉ là lệnh gửi đi mà không biết đã được áp dụng hay chưa.

Phần VI

Ràng buộc, độ tin cậy và phân tích mã nguồn

10 Ràng buộc kỹ thuật và quyết định thiết kế

10.1 Bảng hằng số hệ thống và ý nghĩa vận hành

Để thuận tiện kiểm chứng bằng mã nguồn, bảng sau tổng hợp các hằng số quan trọng đang điều khiển hành vi runtime. Các giá trị cụ thể có thể thay đổi theo phiên bản code, nhưng ý nghĩa kỹ thuật của từng nhóm hằng số cần được giữ ổn định.

Nhóm hằng số	Ví dụ trong code	Ý nghĩa kỹ thuật
Tần suất cảm biến / AI	`SENSOR_INTERVAL_MS`	Chu kỳ cập nhật dữ liệu môi trường và suy luận TinyML ở sender.
Tần suất gửi telemetry	`TELEMETRY_INTERVAL_MS`	Chu kỳ đẩy dữ liệu từ sender sang receiver; ảnh hưởng trực tiếp độ mượt dashboard.
Tần suất snapshot UI	`SNAPSHOT_INTERVAL_MS`	Chu kỳ phát snapshot qua WebSocket; cân bằng giữa realtime và tải CPU/mạng.
Timeout on-line/offline	`OFFLINE_TIMEOUT_MS`	Ngưỡng quyết định sender bị đánh dấu offline khi không còn dữ liệu mới.
Retry cấu hình	`CONFIG_RETRY_MS`	Khoảng chờ trước khi gửi lại packet cấu hình nếu chưa nhận ACK.
Timeout ACK cấu hình	`CONFIG_ACK_TIMEOUT_MS`	Ngưỡng kết luận một phiên đồng bộ cấu hình bị quá hạn.
Retry Wi-Fi / MQTT	`WIFI_RETRY_MS` `MQTT_RETRY_MS`	Tần suất thử kết nối lại hạ tầng mạng và cloud.
Giới hạn số sender/client	`MAX_SENDERS` `MAX_TCP_CLIENTS`	Ràng buộc tài nguyên của receiver; tránh vượt năng lực RAM/socket.

Nhóm hằng số	Ví dụ trong code	Ý nghĩa kỹ thuật
Giới hạn xử lý TCP theo chu kỳ	`TCP_MAX_BYTES_PER_CYCLE` `TCP_MAX_LINES_PER_CYCLE`	Chống một sender chiếm toàn bộ vòng lặp, đảm bảo công bằng cho nhiều sender.

10.2 Trade-off của các quyết định kiến trúc

- **TCP line-delimited JSON thay vì giao thức nhị phân**: dễ debug và dễ đọc khi ghi log; đánh đổi là overhead chuỗi lớn hơn.
- **Receiver giữ vai trò gateway duy nhất**: giúp tập trung điều phối, đồng bộ trạng thái và cloud integration; đánh đổi là receiver trở thành điểm trọng yếu duy nhất.
- **TinyML chạy ở sender**: phản hồi bất thường nhanh tại edge; đánh đổi là firmware sender phức tạp hơn và phải quản lý tài nguyên suy luận.
- **AP+STA đồng thời**: thuận tiện cấu hình cục bộ khi demo; đánh đổi là cần xử lý cẩn thận trạng thái mạng để tránh xung đột luồng.
- **Cơ chế ACK + retry cho cấu hình**: tăng độ tin cậy điều khiển; đánh đổi là state machine phức tạp hơn so với gửi một chiều.

11 Tình huống lỗi, cơ chế phòng vệ và bằng chứng kỹ thuật

11.1 Các tình huống lỗi điển hình

1) Sender mất kết nối tạm thời hoặc reboot giữa chừng.
2) Receiver nhận packet JSON lỗi định dạng hoặc thiếu trường.
3) RPC cloud gửi xuống khi sender đang offline.
4) ACK cấu hình không về đúng hạn do nghẽn mạng nội bộ.
5) Thiết bị lạ gửi telemetry nhưng chưa đăng ký trong danh sách sender chính thức.

11.2 Cơ chế phòng vệ trong mã nguồn

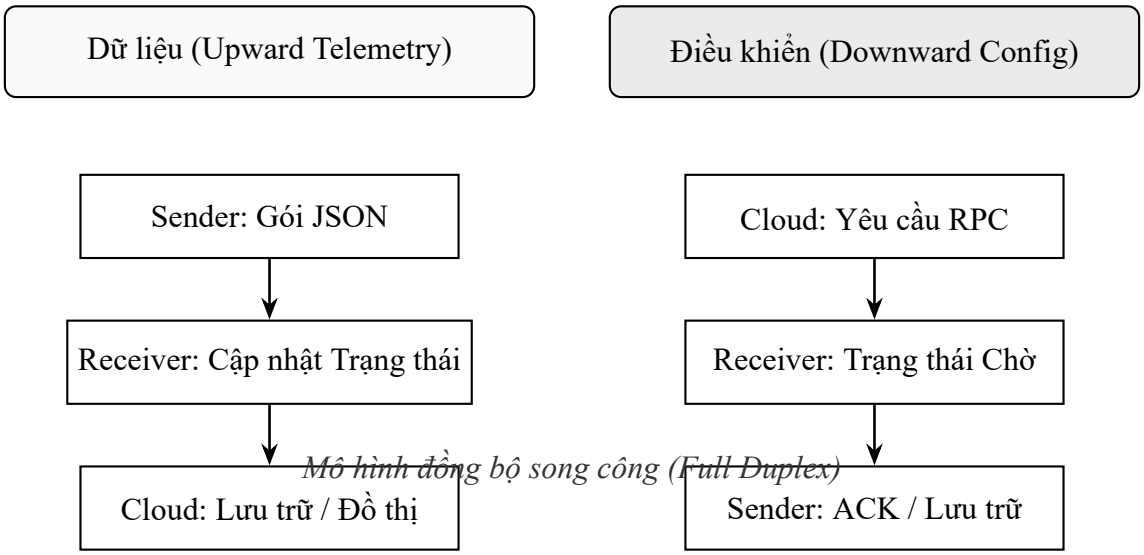
- Parse JSON theo từng dòng và bỏ qua gói lỗi để không làm hỏng vòng lặp chính.
- Tách `known sender` và `unknown device` để tránh tin tưởng thiết bị lạ.
- Chỉ xác nhận đồng bộ cấu hình thành công khi đã nhận `config_ack` hợp lệ.
- Dùng timeout + retry có kiểm soát thay vì gửi lặp vô hạn.

– Giới hạn số byte/số dòng mỗi chu kỳ xử lý TCP để bảo vệ tính công bằng giữa nhiều sender.

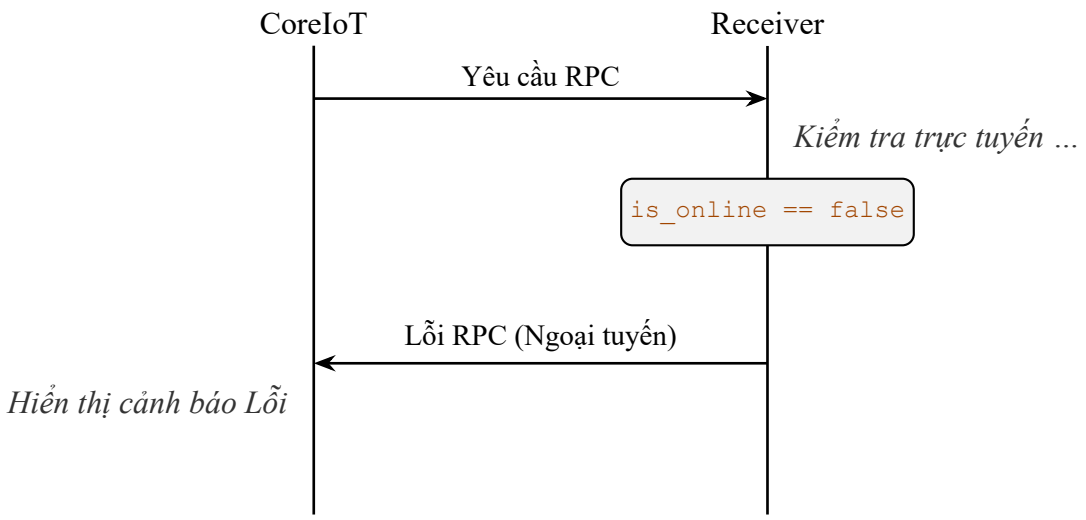
11.3 Bảng đối chiếu luận điểm và bằng chứng mã nguồn

Luận điểm kỹ thuật	Bằng chứng trong mã nguồn	Ý nghĩa
Sender suy luận bất thường tại edge	Hàm `runAiInference()` ở sender	Giảm phụ thuộc cloud, phản hồi trạng thái nhanh tại nguồn dữ liệu.
Receiver điều phối cấu hình hai chiều	`sendConfigPacket()`, `applyConfigAck()`	Đảm bảo lệnh cấu hình được xác nhận thay vì gửi một chiều.
Quản lý liveness bài bản	`handleSender-OfflineTransitions()`	Trạng thái online/offline dựa trên timeout, không phụ thuộc cảm tính.
Tách sender hợp lệ và thiết bị lạ	Nhánh xử lý known/unknown trong transport	Tăng an toàn vận hành và rõ ràng nghiệp vụ.
Cloud chỉ nhìn thấy sender qua gateway	Nhóm hàm MQTT gateway connect/telemetry/rpc	Mô hình triển khai đúng kiến trúc gateway thực tế.

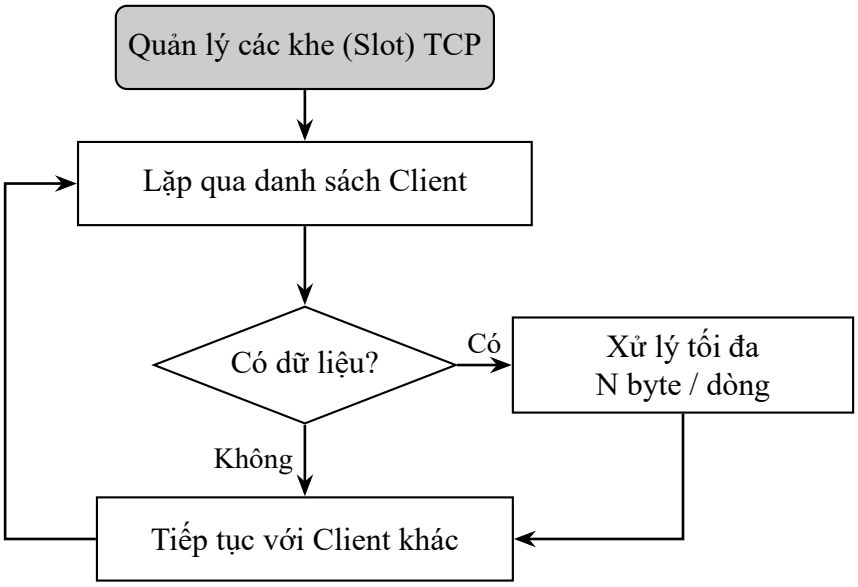
11.4 Ghi chú hình cần chèn ở chương này



Hình 34: Sơ đồ state machine đồng bộ cấu hình: pending, awaiting ACK, timeout, retry, success/fail.



Hình 35: Sequence xử lý lỗi RPC khi sender offline và phản hồi lại cloud.



Hình 36: Flowchart phân bổ slot TCP nhiều sender và cơ chế chống starvation.

12 Phân tích mã nguồn theo module và theo dòng ý nghĩa

12.1 Tổ chức phân tích mã nguồn

Để tránh sao chép dàn trải mã nguồn, phần này chỉ chọn các khối mã có ý nghĩa cao nhất để phân tích. Mỗi trích đoạn mã đều đi kèm ba phần: vai trò của hàm trong hệ thống, giải thích logic xử lý theo bước, và ý nghĩa thiết kế. Cách trình bày này cho phép người đọc hiểu *vì sao hàm tồn tại* và *vì sao nó được viết như vậy*, thay vì chỉ biết hàm đó làm gì.

Điểm mạnh của cấu trúc này là đơn giản và dễ quan sát. Mỗi vòng lặp đều thực hiện các tác vụ cốt lõi, trong đó các hàm con tự kiểm soát bằng timestamp nội bộ để không chạy quá dày. Cách tổ chức như vậy giúp giảm độ phức tạp so với việc tách quá nhiều task FreeRTOS khi chưa thật

sự cần, nhưng vẫn đủ rõ ràng để mở rộng sau này.

12.2 Trích đoạn 2: suy luận TinyML trên sender

```
void runAiInference()
{
    if (g_interpreter == nullptr || g_input == nullptr || g_output
   == nullptr)
    {
        g_aiScore = NAN;
        g_aiStatus = "AI ERROR";
        return;
    }

    g_input->data.f[0] = g_temperature;
    g_input->data.f[1] = g_humidity;

    if (g_interpreter->Invoke() != kTfLiteOk)
    {
        g_aiScore = NAN;
        g_aiStatus = "AI ERROR";
        return;
    }

    g_aiScore = g_output->data.f[0];
    g_aiStatus = g_aiScore >= g_config.threshold ? "ANOMALY" : "
   NORMAL";
}
```

Hàm suy luận TinyML cho thấy đầu vào của mô hình là nhiệt độ và độ ẩm, còn đầu ra là `ai_score` được so sánh với threshold.

Trong đoạn mã này, điều quan trọng không phải là lời gọi `Invoke()` mà là cách sender tự chịu trách nhiệm về tính hợp lệ của inference. Nếu interpreter chưa sẵn sàng hoặc trả về lỗi, sender không cố tiếp tục với giá trị sai mà chuyển hẳn sang trạng thái `AI ERROR`. Đây là lựa chọn hợp lý vì giúp dashboard và receiver phân biệt giữa dữ liệu hợp lệ và lỗi mô hình.

12.3 Trích đoạn 3: gửi telemetry qua TCP

```
void sendTelemetry()
{
    JsonDocument doc;
```

```
    doc["type"] = "telemetry";
    doc["device_id"] = g_deviceId;
    doc["temperature"] = g_temperature;
    doc["humidity"] = g_humidity;
    doc["threshold"] = g_config.threshold;
    doc["ai_score"] = g_aiScore;
    doc["ai_status"] = g_aiStatus;
    doc["latitude"] = g_latitude;
    doc["longitude"] = g_longitude;
    doc["timestamp"] = senderTimestampMs();

    String payload;
    serializeJson(doc, payload);
    payload += '\n';
    g_tcpClient.print(payload);
}
```

Sender dùng JSON newline-delimited để đóng gói telemetry, rất dễ đọc và dễ debug trong quá trình thử nghiệm.

Thiết kế newline-delimited JSON có ưu điểm rõ rệt trong bối cảnh triển khai này. Một mặt, receiver có thể đọc theo từng ký tự và tách gói bằng `\n`. Mặt khác, khi cần quan sát packet thật qua serial hoặc công cụ mạng, người phát triển có thể đọc ngay nội dung mà không cần giải mã định dạng nhị phân.

12.4 Trích đoạn 4: xử lý telemetry ở receiver

```
void applyTelemetry(int slotIndex, const SenderTelemetryPacket &
    packet, const WiFiClient &client)
{
    const int senderIndex = findSenderByDeviceId(String(packet.
    deviceId));
    if (senderIndex < 0)
    {
        markUnknownHeartbeat(String(packet.deviceId), client);
        return;
    }

    SenderState &sender = g_senders[senderIndex];
    sender.online = true;
    sender.hasTelemetry = true;
    sender.lastSeenMs = millis();
    sender.temperature = packet.temperature;
```

```
    sender.humidity = packet.humidity;
    sender.reportedThreshold = packet.threshold;
    sender.aiScore = packet.aiScore;
    sender.aiAnomaly = packet.aiAnomaly;
}
```

Receiver không mặc định chấp nhận mọi thiết bị. Chỉ sender đã đăng ký mới được cập nhật vào `SenderState`; sender lạ sẽ rơi vào nhánh `UnknownDevice`.

Đây là một trong những đoạn mã quan trọng nhất của toàn hệ thống. Nó minh họa đúng tinh thần gateway: trước khi lưu telemetry, receiver luôn ràng buộc thiết bị vào `device_id`. Nhờ bước kiểm tra này, hệ thống tránh được việc một node lạ xuất hiện trong mạng lại ngay lập tức được xem là thiết bị hợp lệ trên cloud.

12.5 Trích đoạn 5: gửi cấu hình từ receiver xuống sender

```
bool sendConfigPacket(int index, uint32_t rpcId)
{
    JsonDocument doc;
    doc["type"] = "config";
    doc["device_id"] = sender.deviceId;
    doc["name"] = sender.name;
    doc["threshold"] = sender.desiredThreshold;
    doc["receiver_ip"] = WiFi.localIP().toString();
    doc["receiver_port"] = LOCAL_TCP_PORT;
    doc["config_version"] = sender.configVersion;

    String payload;
    serializeJson(doc, payload);
    payload += '\n';
    client.socket.print(payload);

    sender.pendingConfig = true;
    sender.awaitingAck = true;
    sender.configSentAtMs = millis();
    return true;
}
```

Receiver gửi packet `config` kèm `configVersion`, rồi chuyển sender sang trạng thái chờ ACK.

Điểm quan trọng ở đây là sau khi gửi cấu hình, receiver không đánh dấu đồng bộ thành công ngay lập tức. Hệ thống chủ động chuyển sang trạng thái `awaitingAck`. Điều này rất quan trọng trong các hệ thống có điều khiển từ xa, bởi vì *lệnh đã gửi* không đồng nghĩa với *thiết bị*

đã áp dụng. Cấu trúc state machine trong hệ thống được xây trên nguyên lý đó.

12.6 Trích đoạn 6: publish RPC response lên CoreIoT

```
void publishRpcResponse(const char *deviceName, uint32_t requestId,
                        bool success, const char *message, float
    threshold)
{
    JsonDocument doc;
    doc["device"] = deviceName;
    doc["id"] = requestId;
    JsonObject data = doc["data"].to<JsonObject>();
    data["success"] = success;
    data["message"] = message;
    if (!std::isnan(threshold))
    {
        data["threshold"] = threshold;
    }
    mqttPublish(MQTT_GATEWAY_RPC_TOPIC, doc);
}
```

RPC response được trả ngược lên CoreIoT sau khi receiver đã biết rõ kết quả cuối cùng từ sender.

Đoạn mã này làm rõ vai trò cầu nối của receiver. Cloud không trực tiếp biết sender nội bộ đã áp dụng cấu hình hay chưa. Chỉ receiver mới tổng hợp được điều đó dựa trên ACK từ sender, rồi phát hành một RPC response có trạng thái thành công hoặc thất bại. Đó chính là giá trị của gateway trong thiết kế này.

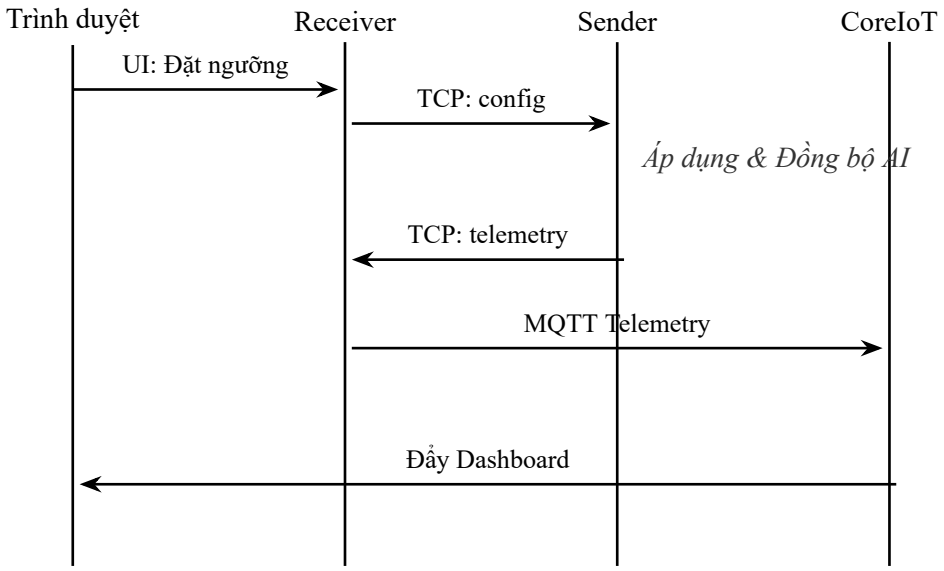
Phần VII

Giao diện, đánh giá và kết luận

13 Kết quả thực nghiệm

13.1 Thiết lập thực nghiệm

Thực nghiệm được thực hiện với hai thiết bị ESP32 đóng vai trò sender (`ESP32-01` và `ESP32-02`) và một ESP32 đóng vai trò receiver, kết nối chung một mạng Wi-Fi nội bộ. Receiver kết nối cloud qua MQTT broker `app.coreiot.io`. Hình 37 tổng hợp toàn bộ luồng dữ liệu và điều khiển đã được xác nhận vận hành đúng trong thực nghiệm: dữ liệu từ sensor đi lên qua TCP tới receiver, tiếp tục lên CoreIoT qua MQTT; chiều ngược lại, lệnh điều khiển từ UI hoặc cloud RPC đi xuống tới từng sender và được xác nhận bằng ACK.



Hình 37: Sequence end-to-end: dữ liệu telemetry đi lên từ sender qua receiver lên cloud, lệnh điều khiển đi ngược chiều và được xác nhận bằng ACK. Đây là luồng đã được xác nhận trong thực nghiệm.

14 Kịch bản thực nghiệm và kết quả đánh giá

14.1 Mục tiêu và kết quả tổng quát

Thực nghiệm nhằm chứng minh bốn nhóm hành vi của hệ thống. Thứ nhất là hành vi bình thường: hai sender cùng hoạt động, gửi telemetry đều đặn, receiver hiển thị đúng và cloud nhận dữ liệu liên tục – đã xác nhận qua ảnh thực nghiệm từ các dashboard cục bộ và CoreIoT. Thứ hai là hành vi cấu hình: khi đổi threshold từ receiver UI hoặc từ cloud RPC, sender nhận packet `config`, áp dụng và trả ACK – đã xác nhận qua trường "Đã đồng bộ" và "Vừa xong" trên

receiver UI. Thứ ba là hành vi lỗi và khôi phục: sender mất kết nối rồi online lại, receiver cập nhật online/offline đúng – được CoreIoT ghi nhận qua alarm *Status Warning*. Thứ tư là hành vi AI: khi dữ liệu vượt ngưỡng, `ai_status` chuyển sang `ANOMALY` và CoreIoT kích hoạt alarm *Humi High* / *Temp High*.

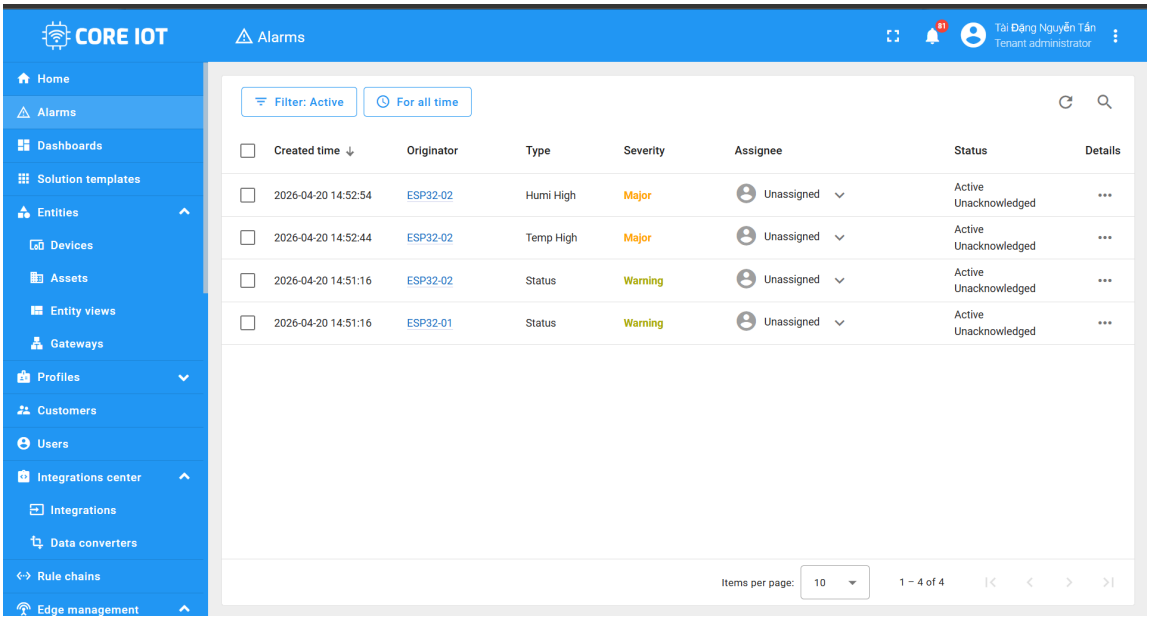
14.2 Danh sách test case và kết quả

TC	Mục tiêu	Bước thực hiện	Kết quả thực tế
TC1	Khởi động hệ thống	Bật receiver, bật các sender và chờ khởi tạo hoàn tất	Hai sender online trên receiver, telemetry liên tục, cloud kết nối ổn định
TC2	Telemetry liên tục	Giữ hệ thống chạy ít nhất 3–5 phút	Receiver nhận dữ liệu tuần tự, CoreIoT có timeline mới, UI cập nhật real-time
TC3	Đổi threshold từ UI	Mở receiver UI, đổi threshold sender và lưu	Sender nhận `config`, cập nhật threshold, trả ACK; receiver hiện "Đã đồng bộ"
TC4	Đổi threshold từ RPC	Gửi RPC `setThreshold` từ CoreIoT	Receiver nhận RPC, gửi packet config, sender ACK, cloud nhận RPC response
TC5	Sender offline	Tắt sender hoặc ngắt Wi-Fi	Receiver đánh dấu offline sau timeout, CoreIoT nhận alarm *Status*
TC6	Sender reconnect	Bật lại sender sau TC5	Receiver nhận telemetry mới, sender trở lại trạng thái online

TC	Mục tiêu	Bước thực hiện	Kết quả thực tế
TC7	AI anomaly	Dữ liệu sensor vượt ngưỡng	`ai_status` chuyển ANOMALY; CoreIoT kích hoạt alarm *Temp High*, *Humi High*
TC8	Unknown device	Sender chưa đăng ký gửi về receiver	Receiver đưa vào danh sách unknown device, không thêm vào sender chính thức

14.3 Bằng chứng thực nghiệm: cảnh báo AI và trạng thái thiết bị

Hình dưới minh họa trực tiếp kết quả TC5 và TC7: CoreIoT ghi nhận 4 alarm đồng thời cho hai sender — *Humi High* và *Temp High* (mức Major) do dữ liệu vượt ngưỡng, và *Status* (mức Warning) do sender thay đổi trạng thái online/offline. Đây là bằng chứng cho thấy toàn bộ pipeline từ TinyML tại sender, qua receiver, lên CoreIoT cloud hoạt động đúng thiết kế.



Hình 38: Hệ thống cảnh báo CoreIoT xác nhận kết quả TC5 (offline/status) và TC7 (AI anomaly).

14.4 Đánh giá ưu điểm của hệ thống

Sau khi hệ thống chạy ổn định ở các test case trên, có thể rút ra một số ưu điểm kỹ thuật. Trước hết, kiến trúc tách sender và receiver làm cho hệ thống có tính mô đun tốt: sender tập trung vào sensing và AI, receiver tập trung vào gateway và cloud. Thứ hai, TinyML được đặt ở sender nên thời gian quyết định bất thường rất ngắn và receiver không phải gánh thêm tính toán. Thứ ba, control plane có ACK và retry, nhờ đó những thay đổi threshold không còn là hành động "gửi rồi hy vọng" mà có xác nhận rõ ràng. Thứ tư, việc có cả Web UI local và cloud giúp hệ thống vừa thuận tiện khi demo tại chỗ vừa có giá trị thực tế khi quan sát từ xa.

14.5 Hạn chế hiện tại

Dù đáp ứng các mục tiêu thiết kế đã đặt ra, hệ thống vẫn còn một số hạn chế cần nêu rõ. Thứ nhất, dữ liệu sensor hiện được mô phỏng nên chưa phản ánh đầy đủ nhiễu và sai số của cảm biến thật. Thứ hai, timestamp trên sender được sinh từ `millis()` cộng với một epoch giả định nên không phải đồng hồ thực chuẩn NTP hay RTC. Thứ ba, cơ chế suy luận địa chỉ receiver theo host `.100` chỉ tối ưu cho subnet `/24`; nếu chuyển sang cấu trúc mạng khác, logic này cần điều chỉnh. Thứ tư, TCP nội bộ chưa có lớp bảo mật và xác thực mạnh; trong môi trường sản phẩm thực tế cần thêm TLS hoặc cơ chế bảo vệ cao hơn.

Phần VIII

Hướng phát triển và kết luận

15 Vì sao kiến trúc này phù hợp bối cảnh triển khai hiện tại

Các hệ IoT triển khai thực tế ngày nay thường không chỉ "đẩy thẳng lên cloud" mà tách rõ ba tầng: xử lý và cảnh báo sớm tại biên, tập trung trạng thái và chính sách ở một gateway cục bộ, rồi mới đồng bộ lên nền tảng đám mây để giám sát và điều khiển từ xa. Kiến trúc nhiều sender – một receiver phản ánh đúng mô hình đó: mỗi sender đóng vai trò điểm đo độc lập (có thể mở rộng thêm node khi cần), còn receiver là đầu mối duy nhất cho kết nối nội bộ, đồng bộ cấu hình và đại diện thiết bị khi làm việc với cloud. Việc đặt TinyML tại sender giảm độ trễ và tải uplink; đường điều khiển hai chiều có xác nhận phù hợp khi ngưỡng hay metadata phải thay đổi an toàn từ giao diện cục bộ hoặc từ cloud. Ở quy mô lắp đặt vừa và nhỏ (văn phòng, phòng lab, khu sản xuất hẹp, nhà kính cỡ vừa), một gateway tích hợp sẵn TCP và MQTT thường đủ dùng mà chưa bắt buộc thêm message broker hay cơ sở dữ liệu riêng; khi nhu cầu tăng, có thể tách tầng lưu trữ hoặc broker mà không phải bỏ lại ý tưởng phân tầng edge – gateway – cloud. Demo với hai sender chỉ là minh họa đa điểm đo; bản thân kiến trúc vẫn giữ được sự vừa đủ về chi phí triển khai và vận hành so với một thiết bị đơn lẻ hay một hệ phân tán quá nặng ngay từ đầu.

16 Khả năng mở rộng trong tương lai

Trên nền tảng đã triển khai, hệ thống có thể được đưa thêm theo nhiều hướng kỹ thuật. Thứ nhất, thay lớp mô phỏng bằng cảm biến thật (ví dụ DHT20, BME280 hoặc cảm biến khí) để kiểm chứng lại độ ổn định của TinyML khi tín hiệu có nhiễu và trôi dạt theo thời gian. Thứ hai, tăng số lượng sender và kiểm tra giới hạn vận hành của receiver khi số kết nối TCP đồng thời tăng (băng thông, RAM, công bằng xử lý vòng lặp). Thứ ba, bổ sung cập nhật firmware qua mạng (OTA) hoặc đồng bộ phiên bản để receiver đóng vai trò điều phối vòng đời thiết bị, không chỉ chuyển tiếp dữ liệu. Thứ tư, củng cố bảo mật tầng ứng dụng và truyền thông: mã hóa, xác thực thiết bị hai chiều, quản lý khóa và chính sách tin cậy. Thứ năm, gắn thêm tầng phân tích và cảnh báo trên cloud (rule engine, ngưỡng động, tích hợp cảnh báo đa kênh) khi nhu cầu vận hành vượt khả năng một dashboard tĩnh.

17 Khả năng áp dụng thực tế

Cùng một khung kiến trúc edge – gateway – cloud có thể được tái sử dụng cho nhiều bài toán triển khai thực tế, chỉ cần điều chỉnh cảm biến, mô hình suy luận và chính sách ngưỡng. Trong nông nghiệp công nghệ cao, mỗi sender có thể là một cụm đo nhiệt độ, độ ẩm và ánh sáng tại từng luống hoặc nhà màng, còn receiver đặt gần điểm phát Wi-Fi để gom dữ liệu và đồng bộ

lên nền tảng giám sát. Trong môi trường công nghiệp nhẹ hoặc phòng lab, các sender thu thập đại lượng tại nhiều vị trí, receiver tập trung cấu hình, phân quyền truy cập và hiển thị trạng thái tại chỗ. Khi thay mô hình TinyML hiện tại bằng mô hình huấn luyện riêng cho từng miền ứng dụng, hệ thống vẫn giữ được luồng cảnh báo sớm tại biên và điều khiển có xác nhận qua gateway, phù hợp làm bản thử nghiệm (pilot) trước khi mở rộng quy mô.

18 Kết luận

Tài liệu đã trình bày quá trình phân tích và hiện thực hệ thống *IoT Application* theo mô hình nhiều sender một receiver, trong đó sender đảm nhiệm mô phỏng cảm biến, thực hiện TinyML tại edge và cung cấp giao diện cục bộ, còn receiver đóng vai trò gateway trung tâm kết nối sender với CoreIoT. Giá trị cốt lõi không nằm ở danh sách tính năng rời rạc mà ở cách tổ chức thành một kiến trúc thống nhất: dữ liệu đi lên qua data plane, cấu hình đi xuống qua control plane, và toàn bộ quá trình có trạng thái rõ ràng để theo dõi, xác nhận và phục hồi.

Phân tích kiến trúc, giao thức, module và mã nguồn tiêu biểu cho thấy hệ thống vượt qua phạm vi một bản demo cảm biến đơn giản và tiệm cận mô hình IoT gateway dùng trong triển khai thực tế. Cách triển khai này phù hợp bối cảnh edge – gateway – cloud hiện nay và là nền tảng để mở rộng quy mô, tính năng bảo mật và tích hợp vận hành trong các bước kế tiếp.

Phần IX

Tài liệu tham khảo

19 Nguồn tham khảo

Bối cảnh và kiến trúc hệ thống

- World Health Organization (WHO). *Immunization in Practice: A Practical Guide for Health Staff — Module 2: The Cold Chain*. Geneva: WHO, 2015. https://www.who.int/publications/i/item/9789241549097
- VNVC. *Nhiệt độ bảo quản vắc xin: 3 tiêu chuẩn nghiêm ngặt*. https://vnvc.vn/nhiet-do-bao-quan-vac-xin/
- W. Shi, J. Cao, Q. Zhang, Y. Li, và L. Xu. "Edge Computing: Vision and Challenges." *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, Oct. 2016. https://doi.org/10.1109/JIOT.2016.2579198

Nền tảng phần cứng

- Espressif Systems. *ESP32 Technical Reference Manual*. Phiên bản 5.2. Shanghai: Espressif Systems, 2024. https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- Espressif Systems. *Arduino core for the ESP32 — ESP32 Arduino*. GitHub, 2016–2024. https://github.com/espressif/arduino-esp32

Học máy tại biên (TinyML)

- P. Warden và D. Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol, CA: O'Reilly Media, 2020. ISBN 978-1-4920-5204-3.
- TensorFlow Authors. *TensorFlow Lite for Microcontrollers*. Google, 2024. https://www.tensorflow.org/lite/microcontrollers

Giao thức và thư viện phần mềm

- OASIS. *MQTT Version 3.1.1 — OASIS Standard*. OASIS, tháng 10 năm 2014. https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html
- I. Fette và A. Melnikov. *The WebSocket Protocol — RFC 6455*. IETF, tháng 12 năm 2011.

https://datatracker.ietf.org/doc/html/rfc6455

– B. Blanchon. *ArduinoJson: An Elegant and Efficient JSON Library for Embedded C++*. 2014–2024. https://arduinojson.org

– LittleFS Project. *littlefs — A little fail-safe filesystem designed for microcontrollers*. GitHub, 2017–2024. https://github.com/littlefs-project/littlefs

Nền tảng IoT cloud

– ThingsBoard, Inc. *MQTT Gateway API Reference*. ThingsBoard Documentation, 2024. https://thingsboard.io/docs/reference/gateway-mqtt-api/

– ThingsBoard, Inc. *ThingsBoard Rule Engine — Alarm Rules*. ThingsBoard Documentation, 2024. https://thingsboard.io/docs/user-guide/alarms/

Phần X

Phụ lục

1 Phụ lục A: Trích đoạn mã nguồn tiêu biểu và diễn giải ngắn

Phụ lục này tập hợp các đoạn mã nguồn tiêu biểu để người đọc đối chiếu nhanh với phần phân tích trong nội dung chính.

A.1 Khởi tạo sender

```
void appSetup()
{
    Serial.begin(115200);
    randomSeed(micros());
    LittleFS.begin(true);
    loadConfig();
    WiFi.mode(WIFI_AP_STA);
    g_deviceId = normalizeMac(WiFi.macAddress());
    g_apSsid = "Sender-" + g_deviceId.substring(g_deviceId.length()
    - 8);
    setupSoftAp();
    maintainWifiSta();
    setupAiEngine();
    startWebServer();
    markReceiverStatus();
}
```

Sender khởi động lần lượt theo đúng các phụ thuộc: file system, config, Wi-Fi, AI và web UI.

A.2 Cấu trúc trạng thái sender

```
struct SenderConfig
{
    char wifiSsid[MAX_TEXT_LEN] = "";
    char wifiPass[MAX_TEXT_LEN] = "";
    char receiverIp[32] = "";
    uint16_t receiverPort = DEFAULT_RECEIVER_PORT;
    float threshold = 0.5f;
    char deviceName[MAX_TEXT_LEN] = "Undefined";
};
```

SenderConfig thể hiện triết lý cấu hình tối giản nhưng đủ cho vận hành thực tế.

A.3 Lưu cấu hình sender xuống LittleFS

```
void saveConfig(bool restartAfterSave)
{
    JsonDocument doc;
    doc["wifiSsid"] = g_config.wifiSsid;
    doc["wifiPass"] = g_config.wifiPass;
    doc["receiverIp"] = g_config.receiverIp;
    doc["receiverPort"] = g_config.receiverPort;
    doc["threshold"] = g_config.threshold;
    doc["deviceName"] = g_config.deviceName;
    File file = LittleFS.open(CONFIG_FILE, "w");
    serializeJson(doc, file);
}
```

Storage trên sender đủ nhỏ gọn để phù hợp với bài toán cấu hình cục bộ.

A.4 Cấu trúc SenderState ở receiver

```
struct SenderState
{
    bool used = false;
    bool registered = false;
    bool online = false;
    bool hasTelemetry = false;
    bool pendingConfig = false;
    bool awaitingAck = false;
    char deviceId[DEVICE_ID_LEN] = "";
    char name[MAX_NAME_LEN + 1] = "";
    float temperature = NAN;
    float humidity = NAN;
    float reportedThreshold = NAN;
    float desiredThreshold = NAN;
    float aiScore = NAN;
    bool aiAnomaly = false;
};
```

Receiver giữ trạng thái phong phú hơn sender vì nó phải điều phối cả data plane lẫn control plane.

A.5 Parse gói telemetry ở receiver

```
bool parseTelemetry(JsonObject data, SenderTelemetryPacket &packet)
```

```
{
    const String deviceId = normalizeDeviceId(String(data["device_id"] | ""));
    copyStringToBuffer(deviceId, packet.deviceId, sizeof(packet.deviceId));
    packet.temperature = data["temperature"] | NAN;
    packet.humidity = data["humidity"] | NAN;
    packet.threshold = data["threshold"] | NAN;
    packet.aiScore = data["ai_score"] | NAN;
    packet.aiAnomaly = String(data["ai_status"] | "").equalsIgnoreCase("ANOMALY");
    packet.timestampMs = data["timestamp"] | 0ULL;
    return true;
}
```

Packet telemetry được parse thành cấu trúc trung gian trước khi hợp nhất vào SenderState.

A.6 Gửi gói cấu hình

```
doc["type"] = "config";
doc["device_id"] = sender.deviceId;
doc["name"] = sender.name;
doc["threshold"] = sender.desiredThreshold;
doc["receiver_ip"] = WiFi.localIP().toString();
doc["receiver_port"] = LOCAL_TCP_PORT;
doc["config_version"] = sender.configVersion;
```

Các trường trong packet cấu hình đủ để sender biết được ngưỡng mới và địa chỉ receiver hiện hành.

A.7 Chủ đề MQTT gateway

```
constexpr char MQTT_GATEWAY_CONNECT_TOPIC[] = "v1/gateway/connect";
constexpr char MQTT_GATEWAY_DISCONNECT_TOPIC[] = "v1/gateway/disconnect";
constexpr char MQTT_GATEWAY_TELEMETRY_TOPIC[] = "v1/gateway/telemetry";
constexpr char MQTT_GATEWAY_ATTRIBUTES_TOPIC[] = "v1/gateway/attributes";
constexpr char MQTT_GATEWAY_RPC_TOPIC[] = "v1/gateway/rpc";
```

Receiver gom toàn bộ giao tiếp cloud vào một nhóm topic nhất quán.

A.8 Snapshot cho giao diện web

```
doc["type"] = "snapshot";
receiver["cloudStatus"] = cloudStatusText();
receiver["wifiConnected"] = WiFi.status() == WL_CONNECTED;
receiver["wifiIp"] = WiFi.localIP().toString();
JsonArray senders = doc["senders"].to<JsonArray>();
JsonArray unknownDevices = doc["unknownDevices"].to<JsonArray>();
```

Snapshot là lớp dữ liệu trung gian quan trọng giữa state nội bộ và UI.